Current Recursion and Tensor-Network Execution in pyAmpliCol

Purpose. This note summarizes the matrix-element generation strategy used by AmpliCol and the corresponding pyAmpliCol implementation. The common idea is to avoid explicit Feynman-diagram enumeration. Instead, matrix elements are assembled from a directed acyclic graph (DAG) of off-shell currents. AmpliCol emits Fortran code that numerically sweeps this current table. pyAmpliCol builds the same logical DAG in Python, lowers bounded pieces to tensor-network expressions handled by Spenso, compiles the resulting parametric scalar/vector expressions with Symbolica evaluators, and can now execute the staged sweep from Rust through the PyO3 runtime `rusticol`.

Current status. The generic LC/NLC/full-colour staged-DAG implementation is the production pyAmpliCol workflow. Process generation writes schema-v2 artifacts loaded by `rusticol`. Run-card arguments may come from TOML, while surviving model parameters are runtime inputs supplied through loader-compatible complex JSON. The default benchmark route is Rusticol with SymJIT stage evaluators and stage-local inputs. Selected-flow JIT artifacts measure stage chunks around base 128 and run at batch 128; all-flow artifacts use uniform chunk 8192 and batch 64. Both use ten Horner iterations, Symbolica's default CPE choice, and enlarged Horner/common-pair limits. The production matrix JIT row uses SymJIT O1 throughout. Repeated ten-second selected-flow checks have not established a consistent, material O3 runtime gain for any current matrix cell, so no O3 substitution is shown; dedicated Z-family scans still report O3/ASM/C++ variants. Selected JIT generation benchmarks effective chunk sizes 64, 96, 128, 192, 256, and unchunked for each stage, retaining a non-default candidate only above a five-percent measured gain. On AArch64, candidate scores count one shared native input pack and sum only evaluator and output-unpack work across chunks, matching Rusticol's fused stage call. Large all-flow and backend-comparison artifacts stay uniformly chunked. The result matrices are generated as separate TeX inputs, and the benchmark writers refresh `pyAmpliCol.pdf` after each completed cell. Generic current and colour-sector generation caps are disabled by default; long benchmark refreshes rely on the 30 GB watchdog rather than truncated colour plans. On macOS the watchdog enforces the larger of process-tree RSS and compressed physical footprint, including descendants that create independent process sessions.

Large runnable artifacts use a compact schema: generation-only current/value metadata is omitted, and stages retain interaction IDs and vertex-kind counts instead of a second copy of every interaction. Evaluator generation lowers, compiles, and writes one stage at a time, then releases its Symbolica expressions before lowering the next stage. Small detailed and plan-only artifacts retain the full records; large diagnostic sidecars explicitly record that they contain the compact runtime schema. Rusticol accepts both layouts and remains backward-compatible with earlier artifacts.

1 The AmpliCol Current Recursion

For a fixed leading-color ordering, a current is labelled by the external subset it contains, the particle type carried by the off-shell line, and any spin/chirality information. Schematically, if I is a set or ordered interval of external colored legs, a current is computed as

$$\mathcal{J}^{\alpha}_{t,\chi}(I) = P^{\alpha}{}_{\beta}(t, p_I) \sum_{\substack{I=A\cup B \\ A\cap B=\emptyset}} \mathcal{V}^{\beta}{}_{\gamma\delta}(t_A, t_B \to t)\, \mathcal{J}^{\gamma}_{t_A,\chi_A}(A)\, \mathcal{J}^{\delta}_{t_B,\chi_B}(B). \tag{1}$$

Here t is the current type (gluon, quark, auxiliary tensor, etc.), χ is the Weyl chirality where relevant, $p_I = \sum_{i\in I} p_i$, P is the propagator, and $\mathcal{V}$ is the appropriate Feynman-rule tensor. The four-gluon vertex is treated through the auxiliary antisymmetric tensor route, so it is still represented by cubic current combinations.

AmpliCol's optimized mode does not loop over full helicity configurations as independent computations. It weaves helicity information into the current table itself. Each external helicity state is a source current with a unique bit in an `ext_cur`-style bitset. An internal current is identified by

$$K(\mathcal{J}) = \big(t, \chi, \mathrm{bin}(I), \mathrm{ext\_cur}\big), \tag{2}$$

where $\mathrm{bin}(I)$ records the external subset and ext_cur is the union of the source-current bits feeding the current. When a new vertex contribution produces the same key as an existing current, AmpliCol appends that vertex to the existing current rather than creating a duplicate node. After all currents are constructed, `curr2amp` records the two endpoint currents whose contraction gives each retained helicity amplitude.

The generated Fortran library has the same layered structure:

```
subroutine evaluate_amp(p, amps)
  complex(kind=8) :: val_c(1:6, n_cur)
  complex(kind=8) :: int_c(1:6, n_vert)

  call compute_external_currents(pp, val_c)
  do size = 2, n_external_minus_one
    call compute_vertices_size(pp, val_c, int_c)
    call compute_currents_size(pp, val_c, int_c)
  end do
  call compute_amps(amps, val_c)
end subroutine
```

The arrays `val_c` and `int_c` are the numerical current and interaction tables. This is the performance-critical idea that pyAmpliCol keeps: one topological sweep produces all retained amplitudes, and shared parent currents are evaluated once.

2 The pyAmpliCol DAG

pyAmpliCol first builds an explicit Python representation of the same current table. In the generic compiler, a current is identified by a model-driven physics index rather than by a process-family name. The identity used for deduplication is

$$K_{\text{pyAmpliCol}} = \big(\text{particle\_id}, L, H, \chi, S, F, Q, C, P, A\big), \tag{3}$$

where L is the sorted tuple of external labels, H is the AmpliCol-style helicity-source ancestry bitset, χ is chirality, S is any spin-state tag needed by the source or propagator, F is flavour flow, Q is charge flow, C is the leading-color state, P is the momentum mask, and A is an optional auxiliary kind such as the antisymmetric tensor used for the four-gluon vertex. The implementation also carries `ordered_external_labels`. That field is leading-color ordering metadata used by the color engine and amplitude grouping, but it is deliberately not part of equality or hashing. Deduplication is therefore exactly equality of Eq. (3); no rule is allowed to ask whether the process is "Z plus jets" or any other family.

field	**role in the current identity**
`particle_id`	Model/UFO particle id, e.g. 21, 23, 1, −1, or an auxiliary id.
`external_labels` and `external_mask`	Which external legs are contained in the current. The mask is the bit representation of the labels.
`helicity_ancestry`	Bitwise union of the source-current helicity bits feeding this current. This is how helicity configurations are woven into one shared DAG.
`chirality` and `spin_state`	Local spin information, especially for Weyl fermions and massive-vector sources.
`flavour_flow` and `charge_flow`	Quantum-number flow checked locally by model vertices, e.g. $u \to dW^+$ or $e^+e^- \to \gamma/Z$.
`color_state`	Colour-ordered open-line or trace state used to build the amplitude vector. LC, NLC and full colour differ in the final sparse colour contraction over this vector.
`momentum_mask`	Which all-outgoing momenta are summed for propagators and momentum-dependent vertices.
`auxiliary_kind`	Optional tag distinguishing auxiliary tensor currents from ordinary particles.

Source currents are built from external leg metadata. Internal currents are then generated by increasing current size, with all interactions that feed the same current index attached to one node.

```
for external helicity source s:
    add_current(key=(pdg_s, labels_s, chirality_s, bit_s), is_source=True)

for current_size in increasing_sizes:
    for compatible split A | B:
        result_key = combine_key(key[A], key[B], vertex_kind)
        result = add_or_reuse_current(result_key)
        add_interaction(left=A, right=B, result=result, vertex=vertex_kind)

for retained helicity ancestry:
    add_amplitude(left=current_with_Z_and_gluons, right=anti_current)

prune_dead_currents_after_helicity_filter()
```

At runtime, pyAmpliCol mirrors the Fortran table layout with contiguous arrays:

$$C_{s,i,c} \equiv \texttt{current\_rows}[s,i,c], \qquad T_{s,v,c} \equiv \texttt{interaction\_rows}[s,v,c],$$

where s is the phase-space sample in the current batch, i is a current id, v is an interaction id, and c is a component index. External wavefunctions fill source-current rows once per sample. Bounded evaluator blocks then fill interactions and derived currents stage by stage:

```
fill_source_currents(current_rows, momenta, helicity_sources)

for stage in current_size_layers:
    state = ParamBuilder.pack(current_rows, interaction_rows, momenta)
    interaction_rows[:, stage.vertices] = vertex_eval[stage](state)
    current_rows[:, stage.currents]     = current_eval[stage](state)

amps = amplitude_eval(ParamBuilder.pack(current_rows, momenta))
me2  = normalization * sum(weight[h] * abs(amps[h])**2 for h in retained_h)
```

This is why the evaluator-only benchmark is meaningful: the expensive recursion is inside compiled evaluator calls, while Python only performs source wavefunction construction, parameter packing, and table transfers.

3 Spenso Tensor Networks and `hep_lib`

Spenso is used to express Lorentz, spinor, auxiliary-tensor, and eventually color contractions in a representation-aware way. A tensor-network expression contains named tensors with typed slots:

$$\mathcal{V}_{\mu\nu}{}^{\rho}(p_A, p_B)\, A^{\mu}\, B^{\nu}, \qquad \text{or in code: } \texttt{V(m("a"),m("b"),m("out")) * A(m("a")) * B(m("b"))}.$$

The expression is not expanded. It is a compact product of tensor factors with shared representation labels. The library hep_lib is the dictionary that tells Spenso what each tensor name means: dense model tensors, metric tensors, momentum-dependent propagators, and parametric rank-one tensors representing runtime inputs.

```
library = model.build_tensor_library()      # returns TensorLibrary.hep_lib_atom()
builder = ParamBuilder()
mink = Representation.mink(4)

builder.register_rank1_tensor(
    library,
    tensor_name="current_A",
    representation=mink,
    head=("stage", "A"),
    length=4,
    role="block_current",
)
builder.register_rank1_tensor(
    library,
    tensor_name="p_A",
    representation=mink,
    head=("stage", "pA"),
    length=4,
    role="block_momentum",
    real_valued=True,
)

raw = V3g(mink("a"), mink("b"), mink("out"), p_A="p_A", p_B="p_B") \
    * TensorName("current_A")(mink("a")).to_expression() \
    * TensorName("current_B")(mink("b")).to_expression()

network = TensorNetwork(raw, library)
network.execute(library=library)
out_tensor = network.result_tensor(library)
evaluator = out_tensor.evaluator(params=builder.parameter_symbols())
```

The important point is that hep_lib stays fixed during the execution. New graph currents are not registered as permanent physics tensors. Instead, the current values are runtime parameters for a bounded block. Spenso executes the factorized tensor network, contracts the representation indices, and returns a parametric output tensor. Symbolica then compiles that parametric tensor to an evaluator. For high multiplicity, pyAmpliCol interleaves this process: after a current or current-size layer is contracted, its numerical output is written back into the current table and becomes an input parameter for later blocks. This avoids one giant scalar expression and keeps the DAG close to AmpliCol's execution model.

The current NLC/full-colour implementation keeps the kinematic DAG colour-ordered and applies a sparse colour matrix to the amplitude vector. For open-line sectors, including arbitrary many quark lines with gluons distributed on the lines, pyAmpliCol contracts the product of left colour strings with the conjugated right strings into closed fundamental traces. Those small colour trace expressions are simplified separately from the kinematic DAG; NLC is obtained by keeping the leading and first $1/N_c^2$-suppressed powers, while full colour keeps the complete sparse metric. If a later implementation puts explicit colour tensors inside a Spenso block, colour simplification still belongs before Spenso execution:

$$\texttt{raw} \leftarrow \texttt{idenso.simplify\_color(raw)}, \qquad \text{then build } \texttt{TensorNetwork(raw, hep\_lib)}.$$

For the present NLC/full-colour milestone, colour algebra is confined to the small colour-basis overlap problem and the Rusticol final contraction, not to expanded kinematic expressions. Fortran AmpliCol is the direct reference for zero, one and two quark-line colour matrices; beyond that, pyAmpliCol's generic open-line metric is validated through raw-amplitude probes and internal colour consistency checks.

4 External UFO and JSON Models

pyAmpliCol keeps its original hard-coded model under the public name `built-in-sm`, but process generation may instead start from a UFO module directory, a JSON serialization produced by `ufo-model-loader`, or an exact-version `*.pyAmplicol-model.json` file. UFO and loader JSON are two front ends to the same object: the loader first produces its Python `Model`, whose algebraic expressions already use Symbolica syntax. A UFO directory is executable Python and is therefore imported in an isolated worker only after the user has accepted it as trusted code. Loader JSON is data-only and bypasses that import step. The conventions follow the UFO 2.0 specification.

Source compilation. The selected restriction (`restrict_default` unless disabled) is applied first, followed by loader simplification. A complete preflight then collects unsupported spins, colour representations, functions, form factors, counterterms, and fermion-flow cases before expensive tensor lowering starts. Particles, parameters, derived couplings, coupling orders, functions, propagators, Lorentz tensors, colour tensors, and vertices are normalized into an immutable `CompiledModel`. Its cache key includes source contents, restriction and simplification choices, the compiler version, and dependency fingerprints. Model conversion is measured once and separately from process generation; the matrix timings start from this compiled representation.

```
env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol inspect-model \
  --model assets/models/json/sm/sm.json

env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol compile-model \
  --model assets/models/ufo/sm \
  --output outputs/sm.pyAmplicol-model.json

env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol generate-process \
  --model outputs/sm.pyAmplicol-model.json 'd d~ > z g g' outputs/dd_zgg
```

Generic kernel preparation. Each UFO colour–Lorentz–coupling term is processed independently; no rule depends on how a particular model author chose tensor names or factorizations. UFO indices are converted to typed Spenso indices, then metric, gamma, Lorentz, and colour identities are simplified with the Symbolica Community Idenso/Spenso routines. The simplified tensor network is materialized only after those algebraic projectors have run. For every possible output leg, pyAmpliCol orients the resulting rule into a binary current kernel and records its parameter names, coupling-order vector, colour weight, source-vertex provenance, and propagator contract.

Kernel components remain named functions while the model is assembled, which keeps the generic lowering compact and auditable. Before each stage evaluator is constructed, small component bodies are inlined so that Symbolica can eliminate common subexpressions across current boundaries. Bodies above 1 KiB remain explicit entries in Symbolica's `functions` map. This prevents spin-2 and high-rank contact tensors from being expanded repeatedly while retaining stable evaluator-call abbreviations; production generation does not depend on Symbolica aliases.

The UFO-declared real/complex types are retained during this step. When a coupling's original UFO expression proves it to be purely real or purely imaginary, the structurally zero component is removed from each stage-local evaluator input; couplings whose phase cannot be proved remain fully complex. This preserves runtime parameter variation while avoiding artificial complex arithmetic introduced only by serialization.

The oriented fermion rules are projected from four-component Dirac tensors to the chiral Weyl basis used by the current table. Colour is retained exactly for full colour; the same simplified generic tensors are projected onto the LC and NLC bases, including open fundamental lines, adjoint words, Fierz terms, and sparse final colour metrics. Explicit UFO propagators are preserved. Otherwise model-driven scalar, fermion, vector, and spin-2 propagators and external wavefunctions are synthesized from particle metadata. A vertex with more than three legs is lowered deterministically into a balanced tree of non-propagating auxiliary currents: there is no fictitious pole, the original coupling is inserted exactly once, and provenance prevents fragments from different contact terms from mixing. An eager n-ary contraction remains the correctness oracle for this decomposition.

From model to process artifact. The compiled particle catalogue resolves process names and multiparticle aliases, crosses incoming legs, and supplies the generic DAG compiler with the oriented kernels above. Reachability and coupling-order pruning happen before current construction; LC/NLC/full projection, helicity ancestry, shared-order recycling, warmup checks, and evaluator compilation then use the same model-independent graph machinery described in the preceding sections. Every process directory embeds the exact compiled model and a complete `model-parameters.json`. Parameters that survive restriction remain runtime-adjustable in Rusticol; a partial complex-JSON override may change masses, widths, or couplings without rebuilding evaluators, while changing a parameter removed by restriction requires recompilation.

For the external-SM matrix comparison, generation also uses `-coupling-order-policy minimal`. This derives the lowest hierarchy-weighted perturbative order from the model's own UFO order metadata and applies the resulting per-order caps. It therefore matches AmpliCol's leading-order workload without a process-family table while leaving ordinary external-model generation at the explicit `all` policy.

5 Example: $d\bar{d} \to Zgg$

This section spells out one concrete DAG so that the split between compiled current contractions and Rust-interpreted scheduling is explicit. The user-level process is

$$d(p_1)\,\bar{d}(p_2) \to Z(p_3)\,g(p_4)\,g(p_5).$$

pyAmpliCol stores the process in the all-outgoing convention. Thus labels $1,\ldots,5$ carry the fields

$$\bar{d}_1, \quad d_2, \quad Z_3, \quad g_4, \quad g_5,$$

with momenta $-p_1$, $-p_2$, p_3, p_4, and p_5, respectively. A label set such as $\{2,4,5\}$ is represented by $L = (2,4,5)$ and by the mask

$$m(2,4,5) = 2^{2-1} + 2^{4-1} + 2^{5-1} = 26.$$

The momentum mask is the same bit pattern in this example, and it tells Rusticol which stored momentum sum must be passed to the next compiled evaluator.

For one leading-color sector, the colored ordered labels are

$$O = (2,4,5,1),$$

with the color-singlet Z_3 allowed to attach anywhere on the quark line. The color engine may also provide the sector $O = (2,5,4,1)$. Nothing in the DAG builder special-cases this as a "$Z + 2g$" process: source currents are ordinary external-leg currents and all later currents are discovered from model vertices.

Source current indices

For a fixed retained source helicity choice, the source entries look as follows. The bit b_i denotes the unique helicity-source bit assigned to that external state.

source	**current**	**mask**	**index payload**
S_1	$\mathcal{J}_{\bar{d}}(\{1\})$	1	$(\text{pid} = -1, L = (1), H = b_1, \chi, S, F = (-1), Q, C_{\bar{q}}, P = 1)$
S_2	$\mathcal{J}_{d}(\{2\})$	2	$(\text{pid} = 1, L = (2), H = b_2, \chi, S, F = (1), Q, C_q, P = 2)$
S_3	$\epsilon_Z(\{3\})$	4	$(\text{pid} = 23, L = (3), H = b_3, S = h_Z, C_{\text{singlet}}, P = 4)$
S_4	$\epsilon_g(\{4\})$	8	$(\text{pid} = 21, L = (4), H = b_4, S = h_4, C_g, P = 8)$
S_5	$\epsilon_g(\{5\})$	16	$(\text{pid} = 21, L = (5), H = b_5, S = h_5, C_g, P = 16)$

The actual stored index also contains the exact leading-color sector id, line-group metadata, and the current dimension. For this example the quark and anti-quark source currents have Weyl-spinor components, gluons and the Z have Lorentz-vector components, and auxiliary tensor currents have the antisymmetric tensor components required by the cubicized four-gluon rule.

What each compiled stage computes

The generic stage compiler groups interactions by the size of the result current. For each group it emits a multi-output Symbolica evaluator. That evaluator is compiled once and receives only parent-current components, momentum sums, and real couplings as parameters. It returns the components of all result currents in the stage.

For the sector $O = (2,4,5,1)$, the schematic stage contents are:

$$E_2: \quad \mathcal{J}_g(4,5) = P_g(p_{45})\, \mathcal{V}_{3g}\big(S_4, S_5; p_4, p_5\big), \tag{4}$$
$$\mathcal{J}_T(4,5) = \mathcal{V}_{ggT}\big(S_4, S_5\big), \tag{5}$$
$$\mathcal{J}_d(2,4) = P_q(p_{24})\, \mathcal{V}_{qgq}\big(S_2, S_4; p_2, p_4\big), \tag{6}$$
$$\mathcal{J}_d(2,3) = P_q(p_{23})\, \mathcal{V}_{qZq}\big(S_2, S_3; p_2, p_3\big), \tag{7}$$

plus the analogous allowed currents for the other color sector and for the anti-quark side. The notation E_2 means a compiled evaluator for all two-label result currents, not a Python function that loops over these formulas component by component.

The next stage consumes the rows written by E_2:

$$E_3: \quad \mathcal{J}_d(2,4,5) = P_q(p_{245})\left[\mathcal{V}_{qgq}\big(\mathcal{J}_d(2,4), S_5\big) + \mathcal{V}_{qgq}\big(S_2, \mathcal{J}_g(4,5)\big)\right], \tag{8}$$
$$\mathcal{J}_d(2,3,4) = P_q(p_{234})\left[\mathcal{V}_{qZq}\big(\mathcal{J}_d(2,4), S_3\big) + \mathcal{V}_{qgq}\big(\mathcal{J}_d(2,3), S_4\big)\right]. \tag{9}$$

The stage compiler does not expand these expressions into diagrams. It collects all interaction records whose output index is the same current and compiles the sum into the output components of that current. If two different splits produce the same current index, their contributions are accumulated inside the same compiled stage output.

The four-label stage then builds the endpoint current used by the closure:

$$E_4: \quad \mathcal{J}_d(2,3,4,5) = P_q(p_{2345}) \sum_{\text{compatible splits}} \mathcal{V}\big(\mathcal{J}_{\text{left}}, \mathcal{J}_{\text{right}}\big). \tag{10}$$

The sum includes, for example, gluon-first and Z-insertion contributions such as

$$\mathcal{V}_{qZq}\big(\mathcal{J}_d(2,4,5), S_3\big), \qquad \mathcal{V}_{qgq}\big(\mathcal{J}_d(2,3,4), S_5\big), \qquad \mathcal{V}_{qgq}\big(\mathcal{J}_d(2,3), \mathcal{J}_g(4,5)\big).$$

Those are still local vertex applications selected by the model and color engine. The result is one stored current row, identified by the full CurrentIndex and by its value-slot range in the artifact.

Finally, a compiled amplitude evaluator returns raw complex roots:

$$E_{\text{amp}}: \quad R_a = \mathcal{V}_{\text{close}}\big(\mathcal{J}_d(2,3,4,5), S_1\big), \tag{11}$$

with one output for each retained root. Roots with the same $(H, \text{color sector}, O)$ coherent-group key are summed before squaring. The reported leading-color matrix element is

$$|\mathcal{M}|^2_{\text{LC}} = \mathcal{N}_{\text{AC}} \sum_{\text{groups } g} w_g \left|\sum_{a \in g} R_a\right|^2. \tag{12}$$

The same example as a DAG

A stripped-down adjacency-list representation of this example is:

```
sources interpreted by Rusticol:
  S1 = J(pid=-1, labels={1}, H=b1, color=qbar-line)
  S2 = J(pid= 1, labels={2}, H=b2, color=q-line)
  S3 = J(pid=23, labels={3}, H=b3, color=singlet)
  S4 = J(pid=21, labels={4}, H=b4, color=line gluon)
  S5 = J(pid=21, labels={5}, H=b5, color=line gluon)

compiled stage E2, subset_size=2:
  I0: S4 + S5 -- V3g, Pg --> J(pid=21, labels={4,5})
  I1: S4 + S5 -- VggT    --> J(aux=T, labels={4,5})
  I2: S2 + S4 -- Vqgq,Pq --> J(pid=1, labels={2,4})
  I3: S2 + S3 -- VqZq,Pq --> J(pid=1, labels={2,3})

compiled stage E3, subset_size=3:
  I4: J(2,4) + S5     -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
  I5: S2     + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
  I6: J(2,4) + S3     -- VqZq,Pq --> J(pid=1, labels={2,3,4})
  I7: J(2,3) + S4     -- Vqgq,Pq --> J(pid=1, labels={2,3,4})

compiled stage E4, subset_size=4:
  I8: J(2,4,5) + S3     -- VqZq,Pq --> J(pid=1, labels={2,3,4,5})
  I9: J(2,3,4) + S5     -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})
  I10:J(2,3)   + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})

compiled amplitude stage:
  A0: J(pid=1, labels={2,3,4,5}) + S1 -- close --> R0
```

This listing is schematic: the real artifact contains all retained helicity ancestries, both leading-color sectors, source dimensions, output component ranges, momentum-slot ids, couplings, and color weights. The important point is the division of labor. The evaluator E_s performs the tensor contractions and the algebraic sum over interactions in stage s. Rusticol interprets the DAG schedule: fill sources, compute momentum sums, call E_2, scatter its outputs into current storage, call E_3, scatter again, and so on until E_{amp} and the final coherent square-sum.

6 Rusticol: the Implemented Rust Runtime

The previous staged pyAmpliCol runtime proved that the shared-current DAG could match Fortran AmpliCol numerically, but its Python loop still moved current tables between many evaluator calls. The current implementation keeps Python as the process generator and validation driver, but moves the hot eager-DAG sweep into a PyO3 extension named `rusticol`. The generated artifact is now a self-contained process directory used by both runtimes:

```python
rt_py = pyamplicol.load_process(process_dir, runtime="python")
values_py = rt_py.evaluate(momenta)

import rusticol
rt = rusticol.Runtime.load(process_dir)
values = rt.evaluate(momenta)
profile = rt.profile(momenta)
```

The Python and Rust runtimes read the same `process_manifest.json`, the same Symbolica evaluator manifests, and the same validation momenta. The Python path is kept as a transparent reference implementation. The Rust path is the performance path.

A process directory contains four classes of data. First, it records process and model metadata: external PDG order, momentum conventions, particle and vertex tables, couplings, masses, symmetry factors, and leading-color normalization. Second, it stores the shared-current DAG: source currents, current ids, dimensions, interaction stages, output slots, retained amplitude records, helicity weights, and color factors. Third, it stores the compiled Symbolica evaluator artifacts for each stage and for the final amplitude/raw-sum stage. Fourth, it contains standalone validation assets: `validation_momenta.json` with decimal-string momenta and a `check_standalone.py` script that imports only `rusticol`, plus `numpy` when available for the optimized f64 input path.

```
# Generated by pyAmpliCol.
process_dir/
  process_manifest.json
  manifest.json
  validation_momenta.json
  check_standalone.py
  compiled/              # C++/ASM shared libraries when present
  ... evaluator states ...
```

`rusticol.Runtime.load` streams the typed schema directly from disk, validates current/value relationships through precomputed indices, loads every Symbolica evaluator, computes direct state offsets for stage outputs, allocates reusable scratch buffers, and builds a fixed execution plan. The optimized f64 path accepts NumPy arrays with shape `(batch, n_external, 4)`. For each batch, Rust fills source wavefunctions and momentum sums, executes stage evaluators in topological order, writes the returned current components into the contiguous state table, executes the amplitude/raw-sum evaluator, applies the Fortran-matched normalization, and returns a NumPy array of matrix elements.

On AArch64, the f64 route loads native two-lane complex SymJIT payloads. A stage-local input slice is packed once and reused by every output chunk in that stage; this keeps chunking as a code-size control without multiplying input-pack cost. Setting `RUSTICOL_NATIVE_SIMD_JIT=0` selects the scalar JIT fallback for diagnostics. Large runnable artifacts likewise omit generation-only current metadata and full per-interaction records after evaluator serialization, retaining compact current/value storage plus stage interaction IDs and per-kind counts. Small or plan-only artifacts may keep the detailed layout. The trusted local diagnostic sidecar is a gzip-compressed Python protocol-5 pickle and identifies which layout it stores; it is not part of evaluator loading or execution.

```
for point in batch {
    fill_source_currents(state_row, point);
    fill_momentum_sums(state_row, point);
}

for stage in stages {
    evaluated = stage.evaluator.evaluate_batch(state);
    assign_stage_outputs(state, evaluated, stage.output_offsets);
}

raw = amplitude_stage.evaluate_raw_sums(state);
me2 = normalization * raw;
```

The `profile` API reports the same split used in the benchmarks: source fill, momentum setup, stage evaluator time, output assignment, amplitude evaluator time, final reduction, total wall time, and memory snapshots where the platform exposes them.

The precision API follows Symbolica's model. `evaluate` and `evaluate_with_prec(..., 16)` use the specialized native f64 path. `evaluate_with_prec(..., 32)` maps the serialized exact evaluator state to Symbolica's `DoubleFloat` coefficients. Other positive precisions map to arbitrary-precision `Float` evaluators. The high-precision routes share the same stage orchestration code through generic Rust traits, while the f64 route remains specialized and avoids high-precision allocation overhead.

The artifact schema is process-generic. The staged execution format no longer needs a Rust-side process-family branch: each process directory supplies the source layout, current slots, stage records, amplitude roots, coherent groups, and evaluator artifacts needed by Rusticol. If a model vertex or color rule is not implemented yet, generation fails with an explicit unsupported-kernel diagnostic rather than falling back to a hidden family path.

7 Shared LC Ordering Recycling

LC process generation now has two related modes. For ordinary integration benchmarks, pyAmpliCol times one selected LC ordering per phase-space point. The selected artifact is pruned before Symbolica evaluator construction, so no dynamic colour-flow selector remains in those expressions. For all-flow fixed-helicity matrix entries, pyAmpliCol writes a materialized shared all-sector artifact: internal currents live in one shared LC sector, while amplitude roots retain the full colour-order mapping used for the AmpliCol `imode=2` comparison. This avoids the old sector-local all-ordering clone while keeping the runtime result directly value-comparable.

The shared all-sector mode reuses pure-gluon subcurrents through the usual LC reflection relation, with the corresponding sign carried as an interaction colour weight. That sign is applied before current-stage propagation, so the shared DAG is numerically equivalent to the sector-specialized artifact while avoiding one current table per ordering. Runtime selection of one LC ordering is still available through generated sidecars; `time-process -lc-sector-ids` loads the matching pruned sidecar when it exists.

The fixed-helicity workload is selected without constructing an all-helicity all-sector artifact. The matrix driver derives replay-compatible candidates from the external source-spin states, evaluates their selected-sector raw sums at a deterministic point, and keeps the strongest nonzero candidate. Exact sector-id lookup is indexed, so this setup remains linear in the number of LC sectors rather than repeatedly scanning the complete colour plan.

Large shared artifacts are serialized with compact current/value metadata and stage interaction IDs. Their evaluators are lowered and compiled stage by stage, so completed Symbolica expressions do not accumulate in memory. For $gg \to t\bar{t} + 5g$, these changes reduced the all-flow O1 build from 122.9 to 71.7 seconds, the process manifest from 376.7 to 45.6 MB, and the complete generated directory from about 435 to 121 MB, without changing the validated result. A bounded $gg \to 8g$ validation constructed all 804,970 currents, 11,371,824 interactions, and 181,440 roots, completed the preceding stage, and entered JIT compilation of the 907,200-output stage at a peak physical footprint of 13.90 GB (11.96 GB RSS). The probe was then stopped deliberately; the earlier non-streamed build had exceeded the 30 GB watchdog while retaining accumulated stage blueprints.

Process	Orders	Curr.	AmpliCol/pyAmpliCol O1 gen	O1 JIT frac.	Sel. ratio	AmpliCol/pyAmpliCol all run	Rel. diff.
$gg \to t\bar{t} + g$	6	29	0.000499 / 0.172	0.328	1.76×	0.658 / 0.628	1.4×10^{-16}
$gg \to t\bar{t} + 2g$	24	118	0.00283 / 0.267	0.587	1.42×	3.61 / 2.54	2.15×10^{-15}
$gg \to t\bar{t} + 3g$	120	592	0.0257 / 1.22	0.772	4.20×	25.7 / 34	0
$gg \to t\bar{t} + 4g$	720	3554	0.304 / 10	0.826	1.11×	198 / 293	4.77×10^{-16}
$gg \to t\bar{t} + 5g$	5040	24880	5.91 / 71.7	0.861	1.37×	2.85×10^3 / 4.33×10^3	6.23×10^{-15}
$gg \to t\bar{t} + 6g$	40320	199042	272 / 1.26×10^3	0.862	1.53×	5.29×10^4 / 5.99×10^4	7.57×10^{-16}

The generation column reports AmpliCol/pyAmpliCol all-flow setup time in seconds for the fixed-helicity `imode=2`-style comparison. All run entries are wall microseconds per phase-space point, AmpliCol/pyAmpliCol. The selected-ratio column is the pyAmpliCol/AmpliCol multiplier for one LC flow with the usual helicity sum; the all-run column is the fixed-source-helicity sum over all LC flows. The generated table is refreshed from the LC matrix cache. At $gg \to t\bar{t} + 6g$, both implementations report 199,042 filtered currents and 40,320 amplitudes/orderings; the selected-flow validation agrees to better than 10^{-15}. The process-matrix tables below use the same selected- and all-flow runtime semantics. Fast changed-path cells and the two memory-study cases were regenerated with the compact writer; validated expensive entries that were unaffected by the writer remain in the cache.

High-multiplicity AArch64 memory limit. For $d\bar{d} \to t\bar{t} + 7g$, the managed SymJIT stack-adjustment patch is still required for frames above the shifted 12-bit ADD/SUB range. The latest upstream branch adds paired complex loads/stores but does not yet split that adjustment. With the patch, all-flow O1 generation progressed to `generic_stage_9_subset_10` before the process tree reached 30.118 GiB. The selected artifact remains valid at 5.4×10^{-14} relative difference; the all-flow slot is marked `>30 GB RAM`, with partial outputs retained.

8 Generic LC Performance Matrix (SymJIT O1)

Cell format: generation time above runtime per phase-space point; slots are AmpliCol, pyAmpliCol SymJIT O1 selected flow, and pyAmpliCol SymJIT O1 all flows. AmpliCol uses generated-library timings. Conventions and gaps are summarized after the table.

ID	base process	n=1				n=2				n=3			
1	$d\bar{d} \to Z + (n-1)g$	2.647 0.022	/ 5.7e-05 / 0.0538	(x0.071) (x6.60)	(x2973) (x1.55)	1.996 0.115	/ 8.4e-05 / 0.121	(x0.113) (x2.69)	(x1713) (x1.20)	1.958 0.424	/ 0.000306 / 0.327	(x0.154) (x1.71)	(x507) (x0.86)
2	$u\bar{d} \to W^+ + (n-1)g$	1.886 0.0119	/ 5.7e-05 / 0.0548	(x0.091) (x9.73)	(x2876) (x1.45)	1.986 0.0717	/ 0.000196 / 0.12	(x0.103) (x3.02)	(x788) (x1.18)	1.938 0.232	/ 0.000304 / 0.326	(x0.13) (x2.11)	(x525) (x0.866)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A				2.016 0.0679	/ 7.8e-05 / 0.126	(x0.102) (x2.64)	(x1944) (x1.00)	1.985 0.154	/ 0.000244 / 0.199	(x0.116) (x2.17)	(x638) (x0.987)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A				2.028 0.0335	/ 7.6e-05 / 0.096	(x0.094) (x3.21)	(x2014) (x1.18)	1.914 0.0613	/ 0.000224 / 0.171	(x0.108) (x3.07)	(x752) (x1.04)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A				2.064 0.165	/ 8.5e-05 / 0.126	(x0.136) (x2.36)	(x1810) (x1.13)	2.026 0.693	/ 0.000245 / 0.259	(x0.23) (x1.46)	(x642) (x0.939)
6	$gg \to t\bar{t} + (n-2)g$	N/A				2.058 0.102	/ 0.000108 / 0.172	(x0.135) (x3.89)	(x1449) (x1.22)	2.283 0.646	/ 0.000499 / 0.658	(x0.176) (x1.76)	(x344) (x0.956)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A				2.025 0.0513	/ 8.5e-05 / 0.124	(x0.098) (x4.22)	(x1799) (x1.45)	2.031 0.234	/ 0.000294 / 0.397	(x0.125) (x2.12)	(x579) (x1.07)
8	$gg \to gg + (n-2)g$	N/A				1.916 0.158	/ 0.000189 / 0.305	(x0.118) (x2.02)	(x838) (x0.843)	2.117 0.855	/ 0.000797 / 1.671	(x0.265) (x1.31)	(x253) (x0.647)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A				N/A				2.424 1.477	/ 0.000271 / 0.385	(x0.379) (x1.42)	(x576) (x0.797)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A				N/A				N/A			
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A				N/A				N/A			
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A				N/A				N/A			
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A				N/A				N/A			
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A				N/A				N/A			

	n=1					n=2					n=3				
gen O1 one-flow, hel. sum	1.89 x0.071	2.65 x0.091	2.27 x0.081	2.27 x0.081	 x0.079	1.92 x0.094	2.06 x0.136	2.01 x0.112	2.02 x0.108	 x0.112	1.91 x0.108	2.42 x0.379	2.08 x0.187	2.03 x0.154	 x0.192
gen O1 all-flows, one-hel	5.7e-5 x2876	5.7e-5 x2973	5.7e-5 x2924	5.7e-5 x2924	 x2924	7.6e-5 x788	0.000196 x2014	0.000113 x1544	8.5e-5 x1756	 x1359	0.000224 x253	0.000797 x752	0.000354 x535	0.000294 x576	 x470
run O1 one-flow, hel. sum	0.0119 x6.60	0.022 x9.73	0.017 x8.16	0.017 x8.16	 x7.70	0.0335 x2.02	0.165 x4.22	0.0955 x3.01	0.0866 x2.85	 x2.79	0.0613 x1.31	1.48 x3.07	0.531 x1.90	0.424 x1.76	 x1.59
run O1 all-flows, one-hel	0.0538 x1.45	0.0548 x1.55	0.0543 x1.50	0.0543 x1.50	 x1.50	0.096 x0.843	0.305 x1.45	0.149 x1.15	0.125 x1.18	 x1.11	0.171 x0.647	1.67 x1.07	0.488 x0.907	0.327 x0.939	 x0.825

Generic LC Performance Matrix (SymJIT O1) (continued)

ID	base process	n=4					n=5					n=6				
1	$d\bar{d} \to Z + (n-1)g$	2.056	/ 0.00306	(x0.369)	(x56.1)		2.32	/ 0.0113	(x0.795)	(x24.4)		3.186	/ 0.107	(x1.28)	(x11.6)	
		1.371	/ 1.236	(x1.36)	(x0.612)		4.356	/ 6.567	(x1.16)	(x0.437)		13.93	/ 41.54	(x0.961)	(x0.755)	
2	$u\bar{d} \to W^+ + (n-1)g$	2.003	/ 0.00161	(x0.247)	(x108)		2.138	/ 0.0117	(x0.57)	(x23.8)		2.495	/ 0.108	(x1.18)	(x11.4)	
		0.864	/ 1.225	(x1.46)	(x0.622)		2.801	/ 6.641	(x1.27)	(x0.432)		9.301	/ 41.73	(x1.02)	(x0.723)	
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.055	/ 0.00101	(x0.174)	(x162)		2.196	/ 0.0074	(x0.264)	(x24.0)		2.493	/ 0.0644	(x0.57)	(x4.47)	
		0.44	/ 0.43	(x1.60)	(x0.815)		1.32	/ 1.434	(x1.28)	(x0.606)		3.995	/ 7.322	(x1.12)	(x0.433)	
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2	/ 0.000976	(x0.146)	(x160)		2.119	/ 0.00723	(x0.19)	(x24.5)		2.208	/ 0.0632	(x0.425)	(x4.47)	
		0.155	/ 0.368	(x2.47)	(x0.915)		0.645	/ 1.279	(x1.53)	(x0.657)		2.123	/ 6.578	(x1.31)	(x0.46)	
5	$d\bar{d} \to ZZ + (n-2)g$	2.319	/ 0.00122	(x0.441)	(x133)		2.578	/ 0.0155	(x0.905)	(x12.7)		3.521	/ 0.0719	(x1.29)	(x5.38)	
		2.073	/ 0.684	(x1.23)	(x0.713)		6.281	/ 2.462	(x0.989)	(x0.522)		17.1	/ 13.35	(x0.937)	(x0.35)	
6	$gg \to t\bar{t} + (n-2)g$	3.189	/ 0.00283	(x0.369)	(x94.1)		11.76	/ 0.0257	(x0.251)	(x47.4)		25.49	/ 0.304	(x0.241)	(x32.9)	
		2.352	/ 3.608	(x1.42)	(x0.704)		8.275	/ 25.72	(x4.20)	(x1.32)		22.51	/ 198	(x1.11)	(x1.48)	
7	$d\bar{d} \to t\bar{t} + (n-2)g$	2.274	/ 0.00148	(x0.229)	(x147)		2.971	/ 0.0114	(x0.439)	(x47.5)		4.123	/ 0.104	(x0.78)	(x31.3)	
		0.864	/ 1.598	(x1.54)	(x0.806)		2.839	/ 10.08	(x1.35)	(x0.527)		8.926	/ 65.32	(x1.11)	(x1.01)	
8	$gg \to gg + (n-2)g$	2.182	/ 0.00655	(x0.597)	(x93.1)		3.267	/ 0.0986	(x0.932)	(x48.0)		5.012	/ 3.495	(x1.38)	(x12.2)	
		3.178	/ 12.73	(x1.22)	(x1.00)		10.91	/ 107	(x1.46)	(x1.40)		32.19	/ 1.06e+03	(x1.22)	(x1.54)	
9	$d\bar{d} \to ZZZ + (n-3)g$	2.97	/ 0.000967	(x0.685)	(x161)		4.709	/ 0.00766	(x0.798)	(x22.7)		8.469	/ 0.0609	(x0.864)	(x4.12)	
		4.665	/ 0.669	(x1.13)	(x0.739)		13	/ 1.655	(x0.91)	(x0.575)		33.08	/ 6	(x0.959)	(x0.413)	
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	3.694	/ 0.00105	(x0.192)	(x156)		4.471	/ 0.00668	(x0.211)	(x25.4)		5.312	/ 0.0563	(x0.289)	(x3.31)	
		0.742	/ 0.587	(x1.99)	(x0.829)		1.581	/ 0.868	(x1.67)	(x0.809)		3.787	/ 1.686	(x1.27)	(x0.683)	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	2.945	/ 0.00114	(x0.218)	(x152)		4.94	/ 0.00754	(x0.31)	(x31.8)		10.1	/ 0.0609	(x0.398)	(x9.01)	
		1.043	/ 0.72	(x1.58)	(x1.06)		3.329	/ 1.907	(x1.36)	(x0.919)		11.48	/ 8.65	(x1.11)	(x0.617)	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	3.879	/ 0.00102	(x0.145)	(x160)		4.611	/ 0.00694	(x0.2)	(x23.6)		5.585	/ 0.0566	(x0.271)	(x3.12)	
		1.394	/ 0.814	(x0.999)	(x0.494)		2.763	/ 1.158	(x0.913)	(x0.486)		6.406	/ 2.127	(x0.71)	(x0.423)	
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	2.188		(x0.107)	0.234		3.18		(x0.102)	0.324		5.499		(x0.129)	0.708	
		0.173		(x1.95)	(N/A)		0.662		(x1.18)	(N/A)		2.289		(x0.845)	(N/A)	
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A					N/A					N/A		2.149	2.149	
												N/A		(5.62)	N/A	
gen O1 one-flow, hel. sum		2|	3.88|	2.6|	2.27		2.12|	11.8|	3.94|	3.18		2.21|	25.5|	6.42|	5.01	
		x0.107|	x0.685|	x0.301|	x0.229|	x0.299	x0.102|	x0.932|	x0.459|	x0.31|	x0.413	x0.129|	x1.38|	x0.699|	x0.57|	x0.542
gen O1 all-flows, one-hel		0.000967|	0.00655|	0.00191|	0.00118		0.00668|	0.0986|	0.0181|	0.0095		0.0563|	3.5|	0.379|	0.0681	
		x56.1|	x162|	x132|	x149|	x112	x12.7|	x48.0|	x29.6|	x24.5|	x38.3	x3.12|	x32.9|	x11.1|	x7.20|	x13.3
run O1 one-flow, hel. sum		0.155|	4.67|	1.49|	1.04		0.645|	13|	4.52|	2.84		2.12|	33.1|	12.9|	9.3	
		x0.999|	x2.47|	x1.53|	x1.46|	x1.32	x0.91|	x4.20|	x1.48|	x1.28|	x1.61	x0.71|	x1.31|	x1.05|	x1.11|	x1.05
run O1 all-flows, one-hel		0.368|	12.7|	2.06|	0.767		0.868|	107|	13.9|	2.18		1.69|	1.06e3|	121|	11	
		x0.494|	x1.06|	x0.776|	x0.773|	x0.868	x0.432|	x1.40|	x0.724|	x0.59|	x1.21	x0.35|	x1.54|	x0.741|	x0.65|	x1.43

Generic LC Performance Matrix (SymJIT O1) (continued)

ID	base process	n=7				n=8				n=9			
1	$d\bar{d} \to Z + (n-1)g$	4.109	/ 1.167	(x1.96)	(x8.17)	8.81	/ 16.39	(x1.89)	(x7.45)	24.92	/ 517	(x1.59)	(x2.70)
		34.92	/ 287	(x1.07)	(x1.06)	90.29	/ 5e+03	(x1.08)	(x0.607)	226	/ 8.89e+04	(x1.32)	(x0.791)
2	$u\bar{d} \to W^+ + (n-1)g$	3.031	/ 1.164	(x2.15)	(x8.42)	4.618	/ 16.03	(x2.89)	(x7.63)	10.29	/ 491	(x2.99)	(x2.81)
		25.54	/ 297	(x1.05)	(x1.03)	68	/ 5.34e+03	(x1.09)	(x0.594)	170	/ 8.44e+04	(x1.40)	(x0.675)
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.67	/ 0.663	(x1.33)	(x1.95)	3.701	/ 6.831	(x2.07)	(x1.47)	7.045	/ 91.97	(x2.21)	(x1.01)
		11.65	/ 45.42	(x0.973)	(x0.645)	32.19	/ 322	(x1.01)	(x0.966)	84.17	/ 6.47e+03	(x1.14)	(x0.567)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2.282	/ 0.653	(x0.968)	(x1.91)	2.655	/ 6.764	(x1.98)	(x1.43)	6.338	/ 101	(x5.82)	(x2.34)
		6.815	/ 42.52	(x1.15)	(x0.673)	20.38	/ 302	(x1.05)	(x1.03)	59.59	/ 6.74e+03	(x1.26)	(x0.726)
5	$d\bar{d} \to ZZ + (n-2)g$	6.452	/ 0.773	(x1.32)	(x2.51)	16.52	/ 7.949	(x1.08)	(x1.97)	27.85	/ 107	(x1.44)	(x1.13)
		43.37	/ 77.24	(x0.956)	(x0.699)	106	/ 539	(x0.968)	(x0.925)	251	/ 1.26e+04	(x0.971)	(x0.439)
6	$gg \to t\bar{t} + (n-2)g$	221	/ 5.905	(x0.059)	(x12.1)	2.3e+03	/ 272	(x0.013)	(x4.63)	N/A			
		59.82	/ 2.85e+03	(x1.37)	(x1.52)	153	/ 5.29e+04	(x1.53)	(x1.13)				
7	$d\bar{d} \to t\bar{t} + (n-2)g$	10.68	/ 1.369	(x0.638)	(x23.4)	70.48	/ 25.68	(x0.219)	(x13.5)	605	/ 1.21e+03	(x0.054)	>30 GB RAM
		25.31	/ 471	(x1.12)	(x1.32)	70.57	/ 9.94e+03	(x1.09)	(x0.81)	170	/ 1.22e+05	(x1.49)	>30 GB RAM
8	$gg \to gg + (n-2)g$	10.5	/ 232	(x1.52)	(x7.55)	51.71	/ 2.48e+04	(x0.799)	>30 GB RAM	N/A			
		82.77	/ 2.62e+04	(x1.25)	(x1.10)	209	/ 3.81e+05	(x1.32)	>30 GB RAM				
9	$d\bar{d} \to ZZZ + (n-3)g$	22.21	/ 0.867	(x0.644)	(x0.744)	66.46	/ 6.915	(x0.481)	(x0.524)	149	/ 80.59	(x0.529)	(x0.372)
		80.67	/ 28.11	(x0.954)	(x0.347)	186	/ 153	(x0.922)	(x0.721)	454	/ 1.4e+03	(x0.86)	(x0.733)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	7.707	/ 0.711	(x0.4)	(x0.351)	14.08	/ 6.051	(x0.467)	(x0.097)	31.49	/ 72.86	(x0.375)	(x0.042)
		9.804	/ 6.02	(x1.06)	(x0.417)	24.15	/ 24.49	(x0.986)	(x0.329)	60.16	/ 132	(x1.08)	(x0.823)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	31.04	/ 0.989	(x0.291)	(x2.39)	134	/ 7.424	(x0.156)	(x2.17)	1.33e+03	/ 81.88	(x0.036)	(x1.05)
		31.54	/ 44.41	(x1.21)	(x1.07)	85.72	/ 263	(x1.22)	(x1.27)	219	/ 2.98e+03	(x1.06)	(x0.904)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	8.688	/ 0.719	(x0.318)	(x0.31)	16.2	/ 6.069	(x0.352)	(x0.076)	37.02	/ 69.15	(x0.301)	(x0.045)
		16.34	/ 7.102	(x0.572)	(x0.266)	36.43	/ 27.39	(x0.577)	(x0.223)	86.55	/ 145	(x0.611)	(x0.723)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	13.03		(x0.128)	1.661	35.07		(x0.117)	4.094	N/A			
		7.384		(x0.682)	(N/A)	21.44		(x0.621)	(N/A)				
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		11.25	11.25	N/A		>30 GB RAM	>30 GB RAM	N/A			
		N/A		(65.4)	N/A	N/A		>30 GB RAM	>30 GB RAM				

Summary	n=7					n=8					n=9				
gen O1 one-flow, hel. sum	2.28	221	26.4	8.69		2.66	2.3e3	209	16.5		6.34	1.33e3	223	29.7	
	x0.059	x2.15	x0.902	x0.644	x0.278	x0.013	x2.89	x0.964	x0.481	x0.08	x0.036	x5.82	x1.53	x0.983	x0.155
gen O1 all-flows, one-hel	0.653	232	20.5	0.928		6.05	2.48e4	2.1e3	7.69		69.1	1.21e3	283	96.5	
	x0.31	x23.4	x5.82	x2.45	x7.63	x0.002	x13.5	x3.41	x1.72	x0.077	x0.027	x2.81	x1.15	x1.03	x1.20
run O1 one-flow, hel. sum	6.82	82.8	33.5	25.5		20.4	209	84.8	70.6		59.6	454	178	170	
	x0.572	x1.37	x1.03	x1.06	x1.10	x0.577	x1.53	x1.04	x1.05	x1.13	x0.611	x1.49	x1.12	x1.11	x1.09
run O1 all-flows, one-hel	6.02	2.62e4	2.53e3	61.3		24.5	3.81e5	3.8e4	431		132	1.22e5	3.26e4	6.6e3	
	x0.266	x1.52	x0.845	x0.867	x1.13	x0.223	x1.27	x0.783	x0.81	x1.01	x0.439	x0.904	x0.709	x0.726	x0.713

Each AmpliCol pair is selected-flow/helicity-summed then all-flow/fixed-helicity; pyAmpliCol SymJIT O1 uses matching pruned and shared artifacts. Entries are generation seconds above runtime wall microseconds per point; pyAmpliCol-only cells are absolute. Green/orange/red means below one/below two/at least two. Summaries are `min|max|avg|med|sum`; `sum` compares paired totals. `>30 GB RAM` marks only the affected selected/all-flow workload. A **VALIDATION FAILED** marker means the same-point difference exceeds the tolerance recorded for that validation result.

Settings. `docs/result_matrix.py` (`-color-accuracy lc`) writes this table and its JSON cache. AmpliCol uses its generated library for the selected workload and the fixed-helicity `imode=2` colour probe for all flows. pyAmpliCol writes separate `selected_flow` and shared `all_flows` artifacts. pyAmpliCol uses Rusticol double precision; selected/all-flow batches are 128/64. Selected JIT stages measure chunks around base 128; all-flow outputs use uniform chunk 8192. Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1, and `-target-runtime 10` are used. The two LC workloads are displayed side by side. The timer-free AmpliCol `imode=2` references are refreshed through $n = 6$; retained $n = 7 \ldots 9$ measurements await the final high-multiplicity pass.

Runtime interpretation. On AArch64, Rusticol runs saved SymJIT stages as native two-lane complex evaluators and packs each stage input once across its chunks. Wall time measures the public `evaluate` call; the profiler separates evaluator and packing work. Low-multiplicity fixed overhead can dominate an AmpliCol call measured in nanoseconds. Ratios at or above two are reviewed throughout, with $n \geq 5$ as the scaling gate.

9 Dedicated $d\bar{d} \to Z+$ Gluon Performance

n	process	route	setup	selected flow, helicity sum gen [s]	wall [us/pt]	eval [us/pt]	all flows, fixed helicity gen [s]	wall [us/pt]	eval [us/pt]
1	d d~ > Z + (1-1)*g	reference	AmpliCol	2.65	0.022	N/A	5.7e-5	0.0538	N/A
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol JIT O1	0.166 (x0.06)	0.136 (x6.19)	0.0357 (x1.62)	0.169 (x2.97e3)	0.0833 (x1.55)	0.0168 (x0.31)
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol ASM O3	0.88 (x0.33)	0.185 (x8.39)	0.09 (x4.09)	0.672 (x1.18e4)	0.0957 (x1.78)	0.0314 (x0.58)
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol C++ O3	0.767 (x0.29)	0.136 (x6.17)	0.0404 (x1.83)	0.722 (x1.27e4)	0.0892 (x1.66)	0.0246 (x0.46)
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol JIT O3	0.179 (x0.07)	0.133 (x6.04)	0.0335 (x1.52)	0.154 (x2.7e3)	0.0778 (x1.45)	0.0159 (x0.29)
2	d d~ > Z + (2-1)*g	reference	AmpliCol	2	0.115	N/A	8.4e-5	0.121	N/A
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol JIT O1	0.264 (x0.13)	0.323 (x2.81)	0.156 (x1.36)	0.164 (x1.95e3)	0.147 (x1.22)	0.0499 (x0.41)
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol ASM O3	1.03 (x0.51)	0.698 (x6.06)	0.542 (x4.70)	1.02 (x1.22e4)	0.241 (x1.99)	0.14 (x1.16)
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol C++ O3	1.26 (x0.63)	0.378 (x3.28)	0.224 (x1.95)	0.992 (x1.18e4)	0.164 (x1.36)	0.0674 (x0.56)
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol JIT O3	0.199 (x0.10)	0.327 (x2.84)	0.155 (x1.35)	0.173 (x2.06e3)	0.143 (x1.18)	0.0492 (x0.41)
3	d d~ > Z + (3-1)*g	reference	AmpliCol	1.96	0.424	N/A	0.000306	0.327	N/A
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol JIT O1	0.233 (x0.12)	0.726 (x1.71)	0.439 (x1.04)	0.172 (x564)	0.287 (x0.88)	0.15 (x0.46)
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol ASM O3	1.38 (x0.70)	1.87 (x4.42)	1.65 (x3.90)	1.23 (x4.01e3)	0.641 (x1.96)	0.507 (x1.55)
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol C++ O3	2.29 (x1.17)	0.936 (x2.21)	0.69 (x1.63)	1.38 (x4.51e3)	0.335 (x1.02)	0.205 (x0.63)
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol JIT O3	0.224 (x0.11)	0.751 (x1.77)	0.454 (x1.07)	0.173 (x565)	0.289 (x0.89)	0.147 (x0.45)
4	d d~ > Z + (4-1)*g	reference	AmpliCol	2.06	1.37	N/A	0.00306	1.24	N/A
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol JIT O1	0.367 (x0.18)	1.9 (x1.39)	1.32 (x0.96)	0.266 (x86.8)	0.766 (x0.62)	0.496 (x0.40)
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol ASM O3	1.8 (x0.87)	5.38 (x3.92)	4.98 (x3.63)	1.55 (x507)	2.18 (x1.76)	1.95 (x1.58)
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol C++ O3	5.48 (x2.67)	2.57 (x1.88)	2.2 (x1.61)	2.35 (x766)	0.981 (x0.79)	0.761 (x0.62)
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol JIT O3	0.406 (x0.20)	1.96 (x1.43)	1.39 (x1.01)	0.192 (x62.7)	0.779 (x0.63)	0.505 (x0.41)
5	d d~ > Z + (5-1)*g	reference	AmpliCol	2.32	4.36	N/A	0.0113	6.57	N/A
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol JIT O1	0.623 (x0.27)	5.54 (x1.27)	4.28 (x0.98)	0.304 (x26.9)	2.9 (x0.44)	2.16 (x0.33)
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol ASM O3	3.88 (x1.67)	16.2 (x3.72)	15 (x3.44)	2.06 (x182)	9.52 (x1.45)	9.1 (x1.39)
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol C++ O3	16.6 (x7.15)	8.07 (x1.85)	7.08 (x1.63)	8.38 (x740)	4.31 (x0.66)	3.8 (x0.58)
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol JIT O3	0.581 (x0.25)	5.62 (x1.29)	4.37 (x1.00)	0.307 (x27.1)	2.94 (x0.45)	2.19 (x0.33)
6	d d~ > Z + (6-1)*g	reference	AmpliCol	3.19	13.9	N/A	0.107	41.5	N/A
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O1	1.14 (x0.36)	13.7 (x0.99)	11.5 (x0.83)	1.34 (x12.5)	32.9 (x0.79)	29 (x0.70)
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol ASM O3	7.37 (x2.31)	46.5 (x3.33)	45 (x3.23)	4.49 (x42)	68.6 (x1.65)	66 (x1.59)
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol C++ O3	46.9 (x14.7)	25.7 (x1.85)	24.3 (x1.75)	71.1 (x665)	57.1 (x1.37)	54.5 (x1.31)
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O3	1.18 (x0.37)	15.5 (x1.11)	13.3 (x0.96)	1.37 (x12.8)	34.6 (x0.83)	31.4 (x0.76)
7	d d~ > Z + (7-1)*g	reference	AmpliCol	4.11	34.9	N/A	1.17	287	N/A
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol JIT O1	2.86 (x0.70)	47.6 (x1.36)	42.9 (x1.23)	10.1 (x8.63)	329 (x1.15)	306 (x1.07)
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol ASM O3	14.8 (x3.61)	126 (x3.62)	121 (x3.47)	21.3 (x18.3)	583 (x2.03)	565 (x1.97)
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol C++ O3	124 (x30.3)	77.4 (x2.22)	73.7 (x2.11)	t/o >15 min	N/A	N/A
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol JIT O3	2.93 (x0.71)	51.8 (x1.48)	45.5 (x1.30)	10 (x8.58)	338 (x1.18)	321 (x1.12)
8	d d~ > Z + (8-1)*g	reference	AmpliCol	8.81	90.3	N/A	16.4	5e3	N/A
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol JIT O1	6.2 (x0.70)	138 (x1.53)	128 (x1.41)	85.6 (x5.22)	3.91e3 (x0.78)	3.76e3 (x0.75)
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol ASM O3	32.5 (x3.69)	323 (x3.57)	324 (x3.58)	158 (x9.65)	6.58e3 (x1.31)	6.37e3 (x1.27)
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol JIT O3	6.28 (x0.71)	142 (x1.57)	131 (x1.45)	85.9 (x5.24)	3.92e3 (x0.78)	3.77e3 (x0.75)
9	d d~ > Z + (9-1)*g	reference	AmpliCol	24.9	226	N/A	517	8.89e4	N/A
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol JIT O1	13.7 (x0.55)	330 (x1.46)	305 (x1.35)	1.4e3 (x2.70)	6.34e4 (x0.71)	6.29e4 (x0.71)
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol ASM O3	70 (x2.81)	858 (x3.79)	843 (x3.72)	1.98e3 (x3.83)	1.21e5 (x1.36)	1.26e5 (x1.41)
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol JIT O3	13.9 (x0.56)	394 (x1.74)	351 (x1.55)	1.4e3 (x2.72)	6.61e4 (x0.74)	7.05e4 (x0.79)

Reproduce the $n = 7$ JIT O1 selected-flow entry with the following generation and timing commands.

```
env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol generate-process 'd d~ > z g g g g g g' docs/.z_performance_outputs/manual/n7/jit_o1 --replace --n_cores 5 --color-accuracy lc --batch-size 64 --symbolica-n-cores 5
      --symbolica-output-chunk-size 128 --symbolica-output-chunk-strategy uniform --symbolica-stage-local-parameter-layout --symbolica-iterations 10 --symbolica-max-horner-scheme-variables 1000 --symbolica-max-common-pair-cache-entries 5000000
      --symbolica-max-common-pair-distance 1000 --lc-sector-ids 0 --reference-color-order 2,4,5,6,7,8,9,1,3 --symbolica-evaluator-backend jit --symbolica-jit-optimization-level 1
env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol time-process --target-runtime 10 --batch-size 64 --json docs/.z_performance_outputs/manual/n7/jit_o1
```

pyAmpliCol model source: `built-in-sm`. Here n is final-state multiplicity. Each block reports generation seconds and wall/evaluator microseconds per point: one selected flow with the helicity sum on the left, all flows at one fixed helicity on the right. Wall measures Rusticol's public `evaluate` call; eval separately profiles evaluator plus input packing. Both use batch 64, stage-local inputs, ten Horner iterations, and a ten-second timing target; output chunks are 128 selected and 8192 all-flow. The green O1 row is the measured default: repeated O3 timings showed no significant Z-family gain. ASM remains a scalar AArch64 diagnostic. The driver keeps generated artifacts under `docs/.z_performance_outputs`. AmpliCol's generated-library time is reported under wall; its separate eval metric is N/A.

10 Generic NLC Performance Matrix

Cell format: generation time above runtime per phase-space point; slots are AmpliCol and pyAmpliCol JIT O1; pyAmpliCol C++ O3 is also shown only through $n = 3$. AmpliCol uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

ID	base process	n=1			n=2			n=3		
1	$d\bar{d} \to Z + (n-1)g$	1.983 0.084	(x0.067) (x1.81) (x0.482)	(x0.475) (x2.29) (x0.606)	2.07 0.228	(x0.075) (x1.44) (x0.68)	(x0.622) (x1.81) (x1.00)	2.137 1.337	(x0.097) (x1.06) (x0.654)	(x1.63) (x1.42) (x1.08)
2	$u\bar{d} \to W^+ + (n-1)g$	2.049 0.0539	(x0.063) (x2.21) (x0.536)	(x0.368) (x3.08) (x0.718)	2.059 0.132	(x0.075) (x1.81) (x0.761)	(x0.552) (x2.26) (x1.06)	2.086 0.742	(x0.084) (x1.24) (x0.729)	(x1.12) (x1.66) (x1.11)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A			2.07 0.178	(x0.078) (x1.12) (x0.428)	(x0.558) (x1.51) (x0.694)	2.125 0.405	(x0.074) (x0.873) (x0.457)	(x0.788) (x1.18) (x0.724)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A			2.043 0.0968	(x0.072) (x1.18) (x0.375)	(x0.515) (x1.91) (x0.684)	2.085 0.185	(x0.071) (x1.14) (x0.46)	(x0.682) (x1.53) (x0.704)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A			2.1 0.324	(x0.075) (x1.29) (x0.595)	(x0.648) (x1.53) (x0.866)	2.152 0.991	(x0.088) (x1.03) (x0.635)	(x1.38) (x1.40) (x1.05)
6	$gg \to t\bar{t} + (n-2)g$	N/A			2.062 0.677	(x0.084) (x1.30) (x0.586)	(x0.883) (x1.52) (x0.854)	2.384 6.593	(x0.179) (x1.37) (x0.836)	(x6.91) (x1.59) (x1.28)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A			2.114 0.479	(x0.075) (x0.96) (x0.331)	(x0.581) (x1.08) (x0.447)	2.337 2.369	(x0.099) (x1.05) (x0.568)	(x1.98) (x1.30) (x0.927)
8	$gg \to gg + (n-2)g$	N/A			2.253 3.216	(x0.089) (x0.442) (x0.294)	(x1.75) (x0.669) (x0.509)	3.166 48.65	(x0.374) (x0.475) (x0.339)	(x23.9) (x0.778) (x0.704)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A			N/A			2.553 1.904	(x0.102) (x1.11) (x0.775)	(x2.32) (x1.55) (x1.26)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A			N/A			N/A		
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A			N/A			N/A		
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A			N/A			N/A		
13	$d\bar{d} \to u\bar{u}\, s\bar{s} + (n-4)g$	N/A			N/A			N/A		
14	$d\bar{d} \to u\bar{u}\, s\bar{s}\, c\bar{c} + (n-6)g$	N/A			N/A			N/A		
summary: gen		1.98 | 2.05 | 2.02 | 2.02 x0.063 | x0.067 | x0.065 | x0.065 | x0.065			2.04 | 2.25 | 2.1 | 2.07 x0.072 | x0.089 | x0.078 | x0.075 | x0.078			2.08 | 3.17 | 2.34 | 2.15 x0.071 | x0.374 | x0.13 | x0.097 | x0.142		
summary: run		0.0539 | 0.084 | 0.0689 | 0.0689 x1.81 | x2.21 | x2.01 | x2.01 | x1.96			0.0968 | 3.22 | 0.666 | 0.276 x0.442 | x1.81 | x1.19 | x1.23 | x0.761			0.185 | 48.6 | 7.02 | 1.34 x0.475 | x1.37 | x1.04 | x1.06 | x0.644		

Generic NLC Performance Matrix (continued)

ID	base process	n=4		n=5		n=6
1	$d\bar{d} \to Z + (n-1)g$	2.588	(x0.282)	4.953	(x1.83)	N/A
		12.05	(x1.12) (x0.775)	154	(x1.47) (x1.20)	
2	$u\bar{d} \to W^+ + (n-1)g$	2.336	(x0.222)	3.727	(x1.61)	N/A
		7.009	(x1.32) (x0.884)	94.09	(x1.72) (x1.39)	
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.24	(x0.09)	3.226	(x0.218)	N/A
		2.062	(x0.646) (x0.419)	17.35	(x0.689) (x0.506)	
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2.14	(x0.078)	2.752	(x0.149)	N/A
		0.97	(x0.687) (x0.412)	8.798	(x0.694) (x0.511)	
5	$d\bar{d} \to ZZ + (n-2)g$	2.337	(x0.18)	4.248	(x0.611)	N/A
		5.508	(x1.04) (x0.727)	48.28	(x1.27) (x1.00)	
6	$gg \to t\bar{t} + (n-2)g$	4.299	(x1.12)	29.13	(x7.67)	N/A
		91.22	(x1.67) (x1.33)	1.5e+03	(x1.83) (x1.58)	
7	$d\bar{d} \to t\bar{t} + (n-2)g$	3.9	(x0.286)	16.75	(x1.15)	N/A
		23.63	(x1.18) (x0.84)	299	(x1.90) (x1.50)	
8	$gg \to gg + (n-2)g$	13.65	(x1.90)	168	(x10.5)	N/A
		828	(x0.862) (x0.762)	1.91e+04	(x0.646) (x0.56)	
9	$d\bar{d} \to ZZZ + (n-3)g$	3.165	(x0.144)	6.75	(x0.272)	N/A
		5.902	(x0.926) (x0.697)	30.76	(x0.99) (x0.824)	
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	4.008	(x0.056)	4.902	(x0.06)	N/A
		1.492	(x1.03) (x0.688)	3.254	(x0.86) (x0.635)	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	2.867	(x0.111)	4.376	(x0.284)	N/A
		3.852	(x0.977) (x0.698)	21.19	(x1.07) (x0.875)	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	4.158	(x0.054)	5.283	(x0.055)	N/A
		2.835	(x0.509) (x0.354)	6.219	(x0.455) (x0.323)	
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A	0.27	N/A	1.385	N/A
		N/A	(3.04)	N/A	(36.7)	
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A		N/A
summary: gen		2.14| 13.7| 3.97| 3.02		2.75| 168| 21.1| 4.93		N/A
		x0.054| x1.90| x0.377| x0.162| x0.737		x0.055| x10.5| x2.04| x0.448| x8.01		N/A
summary: run		0.97| 828| 82| 5.71		3.25| 1.91e4| 1.78e3| 39.5		N/A
		x0.509| x1.67| x0.997| x1.00| x0.951		x0.455| x1.90| x1.13| x1.03| x0.76		N/A

Each cell is raw-library AmpliCol and pyAmpliCol JIT O1; the optional pyAmpliCol C++ O3 slot is shown only through $n = 3$. Entries are generation seconds above runtime microseconds per point; pyAmpliCol runtime shows coloured wall and black pure-evaluator ratios, or absolute time without an AmpliCol reference. Green/orange/red means below one/below two/at least two. Summaries are `min|max|avg|med|sum`; `sum` compares paired totals. A **VALIDATION FAILED** marker means the same-point difference exceeds the tolerance recorded for that validation result.

Settings. `docs/result_matrix.py` (`-color-accuracy nlc`) writes this table and its JSON cache. AmpliCol uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`. One raw evaluator call returns every stored helicity amplitude for a phase-space point; the probe then applies the requested sparse NLC/full colour contractions without reevaluating those amplitudes. pyAmpliCol uses Rusticol double precision, batch size 64, output chunk size 128, Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1, and `-target-runtime 10` are used. C++ O3 is displayed only through $n = 3$; its generation alone is subject to the 15-minute compile cap.

Fortran reference gaps. 2 completed pyAmpliCol cells have no direct Fortran colour reference because that path stops above two quark lines; they are shown as absolute timings.

11 Generic Full-Colour Performance Matrix

Cell format: generation time above runtime per phase-space point; slots are AmpliCol and pyAmpliCol JIT O1; pyAmpliCol C++ O3 is also shown only through $n = 3$. AmpliCol uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

ID	base process	n=1			n=2			n=3		
1	$d\bar{d} \to Z + (n-1)g$	2.046	(x0.065)	(N/A)	2.083	(x0.076)	(N/A)	2.11	(x0.1)	(N/A)
		0.0969	(x1.59) (x0.436)	(N/A)	0.271	(x1.23) (x0.585)	(N/A)	1.546	(x0.948) (x0.597)	(N/A)
2	$u\bar{d} \to W^+ + (n-1)g$	2.027	(x0.067)	(N/A)	2.058	(x0.074)	(N/A)	2.087	(x0.086)	(N/A)
		0.06	(x2.08) (x0.465)	(N/A)	0.147	(x1.69) (x0.703)	(N/A)	0.85	(x1.11) (x0.674)	(N/A)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A			2.091	(x0.072)	(N/A)	2.134	(x0.073)	(N/A)
					0.196	(x1.11) (x0.399)	(N/A)	0.449	(x0.846) (x0.432)	(N/A)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A			2.053	(x0.072)	(N/A)	2.096	(x0.07)	(N/A)
					0.11	(x1.30) (x0.369)	(N/A)	0.203	(x1.04) (x0.432)	(N/A)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A			2.115	(x0.076)	(N/A)	2.151	(x0.088)	(N/A)
					0.384	(x1.23) (x0.559)	(N/A)	1.088	(x0.978) (x0.599)	(N/A)
6	$gg \to t\bar{t} + (n-2)g$	N/A			2.094	(x0.083)	(N/A)	2.385	(x0.178)	(N/A)
					0.807	(x1.04) (x0.516)	(N/A)	8.11	(x1.23) (x0.709)	(N/A)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A			2.084	(x0.075)	(N/A)	2.329	(x0.099)	(N/A)
					0.635	(x0.713) (x0.254)	(N/A)	2.986	(x0.807) (x0.463)	(N/A)
8	$gg \to gg + (n-2)g$	N/A			2.246	(x0.089)	(N/A)	3.133	(x0.402)	(N/A)
					4.149	(x0.389) (x0.236)	(N/A)	63.18	(x0.447) (x0.311)	(N/A)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A			N/A			2.559	(x0.102)	(N/A)
								2.032	(x1.08) (x0.745)	(N/A)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A			N/A			N/A		
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A			N/A			N/A		
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A			N/A			N/A		
13	$d\bar{d} \to u\bar{u}\, s\bar{s} + (n-4)g$	N/A			N/A			N/A		
14	$d\bar{d} \to u\bar{u}\, s\bar{s}\, c\bar{c} + (n-6)g$	N/A			N/A			N/A		
summary: gen		2.03| 2.05| 2.04| 2.04			2.05| 2.25| 2.1| 2.09			2.09| 3.13| 2.33| 2.15		
		x0.065| x0.067| x0.066| x0.066| x0.066			x0.072| x0.089| x0.077| x0.075| x0.077			x0.07| x0.402| x0.133| x0.099| x0.146		
summary: run		0.06| 0.0969| 0.0784| 0.0784			0.11| 4.15| 0.837| 0.328			0.203| 63.2| 8.94| 1.55		
		x1.59| x2.08| x1.84| x1.84| x1.78			x0.389| x1.69| x1.09| x1.17| x0.645			x0.447| x1.23| x0.943| x0.978| x0.583		

Generic Full-Colour Performance Matrix (continued)

ID	base process	n=4		n=5		n=6
1	$d\bar{d} \to Z + (n-1)g$	2.615	(x0.285)	4.981	(x1.92)	N/A
		14.86	(x1.04) (x0.716)	208	(x1.18) (x0.895)	
2	$u\bar{d} \to W^+ + (n-1)g$	2.334	(x0.228)	3.718	(x1.67)	N/A
		8.097	(x1.12) (x0.808)	122	(x1.43) (x1.11)	
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.211	(x0.093)	3.252	(x0.222)	N/A
		2.355	(x0.58) (x0.378)	20.64	(x0.603) (x0.452)	
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2.135	(x0.078)	2.773	(x0.155)	N/A
		1.126	(x0.618) (x0.367)	9.978	(x0.621) (x0.455)	
5	$d\bar{d} \to ZZ + (n-2)g$	2.353	(x0.182)	4.413	(x0.605)	N/A
		6.147	(x0.936) (x0.676)	56.7	(x1.23) (x0.932)	
6	$gg \to t\bar{t} + (n-2)g$	4.316	(x1.13)	28.64	(x8.23)	N/A
		129	(x1.36) (x0.97)	3.05e+03	(x1.20) (x0.82)	
7	$d\bar{d} \to t\bar{t} + (n-2)g$	3.93	(x0.283)	16.78	(x1.16)	N/A
		32.46	(x1.08) (x0.676)	510	(x1.39) (x0.972)	
8	$gg \to gg + (n-2)g$	13.65	(x2.22)	168	(x12.4)	N/A
		1.43e+03	(x0.621) (x0.454)	5.47e+04	(x0.494) (x0.206)	
9	$d\bar{d} \to ZZZ + (n-3)g$	3.121	(x0.143)	6.556	(x0.297)	N/A
		6.077	(x0.915) (x0.68)	32.49	(x1.12) (x0.894)	
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	3.994	(x0.056)	4.835	(x0.062)	N/A
		1.532	(x1.01) (x0.689)	3.345	(x0.851) (x0.637)	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	2.794	(x0.115)	4.391	(x0.287)	N/A
		4.357	(x0.86) (x0.624)	23.72	(x1.12) (x0.899)	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	4.13	(x0.053)	5.311	(x0.055)	N/A
		2.892	(x0.502) (x0.351)	6.453	(x0.412) (x0.312)	
13	$d\bar{d} \to u\bar{u}\, s\bar{s} + (n-4)g$	N/A	0.269	N/A	1.413	N/A
		N/A	(3.38)	N/A	(42.4)	
14	$d\bar{d} \to u\bar{u}\, s\bar{s}\, c\bar{c} + (n-6)g$	N/A		N/A		N/A
summary: gen		2.14| 13.6| 3.97| 2.96		2.77| 168| 21.1| 4.91		N/A
		x0.053| x2.22| x0.405| x0.162| x0.83		x0.055| x12.4| x2.25| x0.451| x9.30		N/A
summary: run		1.13| 1.43e3| 136| 6.11		3.35| 5.47e4| 4.9e3| 44.6		N/A
		x0.502| x1.36| x0.887| x0.925| x0.698		x0.412| x1.43| x0.971| x1.12| x0.544		N/A

Each cell is raw-library AmpliCol and pyAmpliCol JIT O1; the optional pyAmpliCol C++ O3 slot is shown only through $n = 3$. Entries are generation seconds above runtime microseconds per point; pyAmpliCol runtime shows coloured wall and black pure-evaluator ratios, or absolute time without an AmpliCol reference. Green/orange/red means below one/below two/at least two. Summaries are `min|max|avg|med|sum`; `sum` compares paired totals. A **VALIDATION FAILED** marker means the same-point difference exceeds the tolerance recorded for that validation result.

Settings. `docs/result_matrix.py` (`-color-accuracy full`) writes this table and its JSON cache. AmpliCol uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`. One raw evaluator call returns every stored helicity amplitude for a phase-space point; the probe then applies the requested sparse NLC/full colour contractions without reevaluating those amplitudes. pyAmpliCol uses Rusticol double precision, batch size 64, output chunk size 128, Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1, and `-target-runtime 10` are used. C++ O3 is displayed only through $n = 3$; its generation alone is subject to the 15-minute compile cap.

Fortran reference gaps. 2 completed pyAmpliCol cells have no direct Fortran colour reference because that path stops above two quark lines; they are shown as absolute timings.

C++ O3 gaps. 19 optional cells were not run and 0 exceeded the 15-minute compile budget; completed AmpliCol/JIT data remain shown.

Archived validation fixture. All 180 populated LC/NLC/full-colour validation cells through $n \le 9$ are mirrored in `tests/fixtures/result_matrix_references.json`. The fixture contains 94 LC cases and 43 cases for each of NLC and full colour. The integration test checks those cached same-point AmpliCol/pyAmpliCol values without a local Fortran run; `scripts/refresh_result_matrix_fixtures.py` extends the fixture incrementally from the matrix caches.
The following tables repeat the same process families, colour conventions, and AmpliCol references with pyAmpliCol generated from the bundled loader-JSON Standard Model. Numerical validation now covers every populated reference through $n = 9$: all 180 applicable cells agree with the stored built-in/AmpliCol values at identical phase-space points and parameters. The fixture's maximum relative difference is 9.91×10^{-13}. All three colour modes are covered through $n = 5$; the higher-multiplicity campaign follows every populated LC reference, with 13 cases each at $n = 6, 7, 8$ and 10 at $n = 9$. Generation and runtime cells report measured compiled-model runs; the final high-multiplicity campaign is a numerical-validation pass and does not replace those timing measurements. Source conversion is reported separately and is not charged to each process build. Four additional three-quark-line NLC/full-colour timing cells are not among the 180 direct-reference comparisons because Fortran AmpliCol does not expose that colour contraction; they are labelled accordingly in the generated tables. Blank cells are not copied built-in measurements. Majorana/FNV interactions, spin 3/2, sextet external states, loop/counterterm vertices, and opaque callbacks remain explicit preflight errors in this milestone; massive spin-2 support remains experimental.

External-SM shared-LC summary

Process	Orders	Curr.	AmpliCol/pyAmpliCol O1 gen	O1 JIT frac.	Sel. ratio	AmpliCol/pyAmpliCol all run	Rel. diff.
$gg \to t\bar{t} + g$	6	29	0.000499 / 0.229	0.307	1.78×	0.658 / 0.646	1.96×10^{-15}
$gg \to t\bar{t} + 2g$	24	118	0.00283 / 0.405	0.453	1.60×	3.61 / 2.79	4.62×10^{-15}
$gg \to t\bar{t} + 3g$	120	592	0.0257 / 1.68	0.741	1.15×	25.7 / 33.3	1.48×10^{-15}
$gg \to t\bar{t} + 4g$	720	3554	0.304 / 14.1	0.782	1.42×	198 / 344	3.46×10^{-15}

Reproducing the numerical fixture. The validation driver regenerates every applicable LC/NLC/full-colour case at the exact archived phase-space point and with a matched runtime parameter card. For the reduced observable it requires

$$\frac{|M_{\mathrm{UFO}} - M_{\mathrm{ref}}|}{\max(|M_{\mathrm{UFO}}|, |M_{\mathrm{ref}}|, 10^{-300})} \le 10^{-10}.$$

It writes the cumulative report after every case and preserves superseded process directories in the same output tree. The committed incremental reports are `docs/ufo_sm_n4_validation_report.json` through `docs/ufo_sm_n9_validation_report.json`.

```
env PYTHONPATH=src dependencies/.venv/bin/python \
  scripts/run_with_memory_watch.py --limit-gb 30 -- \
  dependencies/.venv/bin/python scripts/validate_ufo_sm_fixture.py \
  --min-n 1 --max-n 9 --color-accuracy all --n-cores 5 \
  --tolerance 1e-10 --regenerate --keep-going \
  --output-root .ufo_support_outputs/ufo_sm_fixture_validation_n9
```

12 External-SM LC Performance Matrix (SymJIT O1)

pyAmpliCol model source: **external UFO/JSON SM**. Cell format: generation time above runtime per phase-space point; slots are AmpliCol, pyAmpliCol SymJIT O1 selected flow, and pyAmpliCol SymJIT O1 all flows. AmpliCol uses generated-library timings. Conventions and gaps are summarized after the table.

ID	base process	n=1				n=2				n=3			
1	$d\bar{d} \to Z + (n-1)g$	2.647 0.022	/5.7e-05 /0.0538	(x0.076) (x6.73)	(x3554) (x1.62)	1.996 0.115	/8.4e-05 /0.121	(x0.113) (x2.94)	(x2622) (x1.30)	1.958 0.424	/0.000306 /0.327	(x0.115) (x1.87)	(x703) (x0.908)
2	$u\bar{d} \to W^+ + (n-1)g$	1.886 0.0119	/5.7e-05 /0.0548	(x0.112) (x10.2)	(x3674) (x1.55)	1.986 0.0717	/0.000196 /0.12	(x0.188) (x3.37)	(x2202) (x1.28)	1.938 0.232	/0.000304 /0.326	(x0.106) (x2.22)	(x639) (x0.928)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A				2.016 0.0679	/7.8e-05 /0.126	(x0.111) (x2.93)	(x2751) (x1.10)	1.985 0.154	/0.000244 /0.199	(x0.103) (x2.34)	(x863) (x1.03)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A				2.028 0.0335	/7.6e-05 /0.096	(x0.106) (x3.57)	(x2746) (x1.32)	1.914 0.0613	/0.000224 /0.171	(x0.1) (x3.37)	(x847) (x1.16)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A				2.064 0.165	/8.5e-05 /0.126	(x0.234) (x3.64)	(x5101) (x2.50)	2.026 0.693	/0.000245 /0.259	(x0.133) (x1.53)	(x897) (x1.00)
6	$gg \to t\bar{t} + (n-2)g$	N/A				2.058 0.102	/0.000108 /0.172	(x0.137) (x4.10)	(x2142) (x1.40)	2.283 0.646	/0.000499 /0.658	(x0.116) (x1.78)	(x459) (x0.983)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A				2.025 0.0513	/8.5e-05 /0.124	(x0.182) (x6.09)	(x3855) (x1.85)	2.031 0.234	/0.000294 /0.397	(x0.108) (x2.17)	(x740) (x1.10)
8	$gg \to gg + (n-2)g$	N/A				1.916 0.158	/0.000189 /0.305	(x0.124) (x2.19)	(x1212) (x0.906)	2.117 0.855	/0.000797 /1.671	(x0.122) (x1.37)	(x339) (x0.66)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A				N/A				2.424 1.477	/0.000271 /0.385	(x0.299) (x1.46)	(x808) (x0.804)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A				N/A				N/A			
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A				N/A				N/A			
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A				N/A				N/A			
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A				N/A				N/A			
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A				N/A				N/A			
gen O1 one-flow, hel. sum		1.89| 2.65| 2.27| 2.27 x0.076| x0.112| x0.094| x0.094| x0.091				1.92| 2.06| 2.01| 2.02 x0.106| x0.234| x0.15| x0.131| x0.15				1.91| 2.42| 2.08| 2.03 x0.1| x0.299| x0.134| x0.115| x0.137			
gen O1 all-flows, one-hel		5.7e-5| 5.7e-5| 5.7e-5| 5.7e-5 x3554| x3674| x3614| x3614| x3614				7.6e-5|0.000196|0.000113| 8.5e-5 x1212| x5101| x2829| x2684| x2549				0.000224|0.000797|0.000354|0.000294 x339| x897| x699| x740| x617			
run O1 one-flow, hel. sum		0.0119| 0.022| 0.017| 0.017 x6.73| x10.2| x8.45| x8.45| x7.93				0.0335| 0.165| 0.0955| 0.0866 x2.19| x6.09| x3.60| x3.47| x3.37				0.0613| 1.48| 0.531| 0.424 x1.37| x3.37| x2.01| x1.87| x1.66			
run O1 all-flows, one-hel		0.0538| 0.0548| 0.0543| 0.0543 x1.55| x1.62| x1.58| x1.58| x1.58				0.096| 0.305| 0.149| 0.125 x0.906| x2.50| x1.46| x1.31| x1.38				0.171| 1.67| 0.488| 0.327 x0.66| x1.16| x0.953| x0.983| x0.855			

External-SM LC Performance Matrix (SymJIT O1) (continued)

ID	base process	n=4				n=5				n=6			
1	$d\bar{d} \to Z + (n-1)g$	2.056	/ 0.00306	(x0.154)	(x73.8)	2.32	/ 0.0113	(x0.856)	(x36.8)	3.186	/ 0.107	(x1.55)	(x14.8)
		1.371	/ 1.236	(x1.40)	(x0.626)	4.356	/ 6.567	(x1.28)	(x0.461)	13.93	/ 41.54	(x1.14)	(x0.816)
2	$u\bar{d} \to W^+ + (n-1)g$	2.003	/ 0.00161	(x0.138)	(x140)	2.138	/ 0.0117	(x0.415)	(x32.7)	2.495	/ 0.108	(x1.35)	(x14.7)
		0.864	/ 1.225	(x1.64)	(x0.636)	2.801	/ 6.641	(x1.34)	(x0.464)	9.301	/ 41.73	(x1.16)	(x0.815)
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.055	/ 0.00101	(x0.114)	(x198)	2.196	/ 0.0074	(x0.163)	(x34.1)	2.493	/ 0.0644	(x0.397)	(x6.17)
		0.44	/ 0.43	(x1.73)	(x0.852)	1.32	/ 1.434	(x1.42)	(x0.649)	3.995	/ 7.322	(x1.19)	(x0.457)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2	/ 0.000976	(x0.112)	(x202)	2.119	/ 0.00723	(x0.119)	(x33.4)	2.208	/ 0.0632	(x0.284)	(x5.97)
		0.155	/ 0.368	(x2.58)	(x0.955)	0.645	/ 1.279	(x1.63)	(x0.683)	2.123	/ 6.578	(x1.36)	(x0.477)
5	$d\bar{d} \to ZZ + (n-2)g$	2.319	/ 0.00122	(x0.421)	(x175)	2.578	/ 0.0155	(x1.06)	(x16.6)	3.521	/ 0.0719	(x1.71)	(x7.63)
		2.073	/ 0.684	(x1.44)	(x0.736)	6.281	/ 2.462	(x1.18)	(x0.554)	17.1	/ 13.35	(x1.27)	(x0.385)
6	$gg \to t\bar{t} + (n-2)g$	3.189	/ 0.00283	(x0.291)	(x143)	11.76	/ 0.0257	(x0.289)	(x65.6)	25.49	/ 0.304	(x0.342)	(x46.5)
		2.352	/ 3.608	(x1.60)	(x0.773)	8.275	/ 25.72	(x1.15)	(x1.29)	22.51	/ 198	(x1.42)	(x1.73)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	2.274	/ 0.00148	(x0.138)	(x208)	2.971	/ 0.0114	(x0.371)	(x74.4)	4.123	/ 0.104	(x0.898)	(x48.3)
		0.864	/ 1.598	(x1.59)	(x0.839)	2.839	/ 10.08	(x1.41)	(x0.545)	8.926	/ 65.32	(x1.70)	(x1.27)
8	$gg \to gg + (n-2)g$	2.182	/ 0.00655	(x0.675)	(x137)	3.267	/ 0.0986	(x0.974)	(x63.0)	5.012	/ 3.495	(x1.69)	(x15.6)
		3.178	/ 12.73	(x1.24)	(x1.07)	10.91	/ 107	(x1.44)	(x1.41)	32.19	/ 1.06e+03	(x0.943)	(x1.47)
9	$d\bar{d} \to ZZZ + (n-3)g$	2.97	/ 0.000967	(x0.684)	(x210)	4.709	/ 0.00766	(x0.958)	(x31.6)	8.469	/ 0.0609	(x1.13)	(x6.43)
		4.665	/ 0.669	(x1.19)	(x0.758)	13	/ 1.655	(x1.21)	(x0.595)	33.08	/ 6	(x1.33)	(x0.434)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	3.694	/ 0.00105	(x0.085)	(x202)	4.471	/ 0.00668	(x0.142)	(x33.7)	5.312	/ 0.0563	(x0.219)	(x4.47)
		0.742	/ 0.587	(x2.16)	(x0.89)	1.581	/ 0.868	(x1.82)	(x0.835)	3.787	/ 1.686	(x1.39)	(x0.705)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	2.945	/ 0.00114	(x0.119)	(x222)	4.94	/ 0.00754	(x0.321)	(x44.6)	10.1	/ 0.0609	(x0.542)	(x12.5)
		1.043	/ 0.72	(x1.74)	(x1.10)	3.329	/ 1.907	(x1.48)	(x0.946)	11.48	/ 8.65	(x1.22)	(x0.679)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	3.879	/ 0.00102	(x0.089)	(x239)	4.611	/ 0.00694	(x0.144)	(x33.7)	5.585	/ 0.0566	(x0.205)	(x4.80)
		1.394	/ 0.814	(x1.08)	(x0.652)	2.763	/ 1.158	(x0.969)	(x0.677)	6.406	/ 2.127	(x0.805)	(x0.606)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	2.188		(x0.097)	0.213	3.18		(x0.083)	0.265	5.499		(x0.089)	0.49
		0.173		(x2.02)	(N/A)	0.662		(x1.23)	(N/A)	2.289		(x0.915)	(N/A)
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A				N/A				N/A			

	n=4					n=5					n=6				
gen O1 one-flow, hel. sum	2	3.88	2.6	2.27		2.12	11.8	3.94	3.18		2.21	25.5	6.42	5.01	
	x0.085	x0.684	x0.24	x0.138	x0.237	x0.083	x1.06	x0.453	x0.321	x0.42	x0.089	x1.71	x0.8	x0.542	x0.655
gen O1 all-flows, one-hel	0.000967	0.00655	0.00191	0.00118		0.00668	0.0986	0.0181	0.0095		0.0563	3.5	0.379	0.0681	
	x73.8	x239	x179	x200	x156	x16.6	x74.4	x41.7	x33.9	x52.1	x4.47	x48.3	x15.7	x10.1	x17.5
run O1 one-flow, hel. sum	0.155	4.67	1.49	1.04		0.645	13	4.52	2.84		2.12	33.1	12.9	9.3	
	x1.08	x2.58	x1.65	x1.60	x1.42	x0.969	x1.82	x1.35	x1.34	x1.29	x0.805	x1.70	x1.22	x1.22	x1.22
run O1 all-flows, one-hel	0.368	12.7	2.06	0.767		0.868	107	13.9	2.18		1.69	1.06e3	121	11	
	x0.626	x1.10	x0.824	x0.806	x0.929	x0.461	x1.41	x0.759	x0.663	x1.22	x0.385	x1.73	x0.82	x0.692	x1.43

Each AmpliCol pair is selected-flow/helicity-summed then all-flow/fixed-helicity; pyAmpliCol SymJIT O1 uses matching pruned and shared artifacts. Entries are generation seconds above runtime wall microseconds per point; pyAmpliCol-only cells are absolute. The pyAmpliCol entries use the compiled `external UFO/JSON SM` source; the AmpliCol columns are the same matched external reference used by the built-in-SM table. Green/orange/red means below one/below two/at least two. Summaries are `min|max|avg|med|sum`; `sum` compares paired totals. `>30 GB RAM` marks only the affected selected/all-flow workload. A **VALIDATION FAILED** marker means the same-point difference exceeds the tolerance recorded for that validation result.

Settings. `docs/result_matrix.py` (`-color-accuracy lc`) writes this table and its JSON cache. AmpliCol uses its generated library for the selected workload and the fixed-helicity `imode=2` colour probe for all flows. pyAmpliCol writes separate `selected_flow` and shared `all_flows` artifacts. pyAmpliCol uses Rusticol double precision; selected/all-flow batches are 128/64. Selected JIT stages measure chunks around base 128; all-flow outputs use uniform chunk 8192. Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1, and `-target-runtime 10` are used. The two LC workloads are displayed side by side. The timer-free AmpliCol `imode=2` references are refreshed through $n = 6$; retained $n = 7 \ldots 9$ measurements await the final high-multiplicity pass.

Runtime interpretation. On AArch64, Rusticol runs saved SymJIT stages as native two-lane complex evaluators and packs each stage input once across its chunks. Wall time measures the public `evaluate` call; the profiler separates evaluator and packing work. Low-multiplicity fixed overhead can dominate an AmpliCol call measured in nanoseconds. Ratios at or above two are reviewed throughout, with $n \geq 5$ as the scaling gate.

13 External-SM Dedicated $d\bar{d} \to Z+$ Gluon Performance

n	process	route	setup	selected flow, helicity sum gen [s]	vs blt-in	wall [us/pt]	vs blt-in	eval [us/pt]	all flows, fixed helicity gen [s]	vs blt-in	wall [us/pt]	vs blt-in	eval [us/pt]
1	d d~ > Z + (1-1)*g	reference	AmpliCol	1.87		0.0193		N/A	0.101		0.859		N/A
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol JIT O1	0.229 (x0.12)	x1.38	0.146 (x7.56)	x1.07	0.0381 (x1.97)	0.182 (x1.79)	x1.07	0.0831 (x0.10)	x1.00	0.0171 (x0.02)
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol ASM O3	1.02 (x0.54)	x1.16	0.212 (x11)	x1.15	0.113 (x5.85)	1.04 (x10.3)	x1.55	0.107 (x0.12)	x1.11	0.0355 (x0.04)
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol C++ O3	1.23 (x0.66)	x1.60	0.152 (x7.86)	x1.12	0.0467 (x2.42)	1.12 (x11.1)	x1.55	0.106 (x0.12)	x1.19	0.0295 (x0.03)
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol JIT O3	0.24 (x0.13)	x1.34	0.144 (x7.45)	x1.08	0.0363 (x1.88)	0.187 (x1.84)	x1.21	0.0858 (x0.10)	x1.10	0.0165 (x0.02)
2	d d~ > Z + (2-1)*g	reference	AmpliCol	2		0.115		N/A	0.102		0.968		N/A
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol JIT O1	0.288 (x0.14)	x1.09	0.34 (x2.96)	x1.05	0.161 (x1.40)	0.206 (x2.02)	x1.25	0.148 (x0.15)	x1.01	0.0493 (x0.05)
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol ASM O3	1.79 (x0.90)	x1.75	0.947 (x8.22)	x1.36	0.739 (x6.41)	1.36 (x13.4)	x1.33	0.3 (x0.31)	x1.25	0.194 (x0.20)
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol C++ O3	1.82 (x0.91)	x1.45	0.415 (x3.60)	x1.10	0.235 (x2.04)	1.56 (x15.3)	x1.57	0.186 (x0.19)	x1.13	0.084 (x0.09)
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol JIT O3	0.252 (x0.13)	x1.27	0.388 (x3.36)	x1.19	0.178 (x1.54)	0.205 (x2.02)	x1.18	0.151 (x0.16)	x1.06	0.0525 (x0.05)
3	d d~ > Z + (3-1)*g	reference	AmpliCol	1.96		0.424		N/A	0.0944		1.14		N/A
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol JIT O1	0.403 (x0.21)	x1.73	0.826 (x1.95)	x1.14	0.497 (x1.17)	0.236 (x2.50)	x1.37	0.318 (x0.28)	x1.11	0.16 (x0.14)
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol ASM O3	2.46 (x1.26)	x1.79	4.62 (x10.9)	x2.47	2.39 (x5.63)	2.3 (x24.4)	x1.88	1.4 (x1.23)	x2.19	0.684 (x0.60)
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol C++ O3	2.97 (x1.52)	x1.30	1.03 (x2.43)	x1.10	0.779 (x1.84)	1.96 (x20.8)	x1.42	0.382 (x0.33)	x1.14	0.231 (x0.20)
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol JIT O3	0.466 (x0.24)	x2.08	0.825 (x1.95)	x1.10	0.503 (x1.19)	0.226 (x2.39)	x1.30	0.32 (x0.28)	x1.10	0.159 (x0.14)
4	d d~ > Z + (4-1)*g	reference	AmpliCol	2.06		1.37		N/A	0.0961		1.95		N/A
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol JIT O1	0.636 (x0.31)	x1.73	2 (x1.46)	x1.05	1.4 (x1.02)	0.281 (x2.92)	x1.06	0.797 (x0.41)	x1.04	0.499 (x0.26)
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol ASM O3	2.38 (x1.16)	x1.32	6.15 (x4.49)	x1.14	5.75 (x4.19)	1.84 (x19.1)	x1.18	2.3 (x1.18)	x1.06	2.06 (x1.06)
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol C++ O3	6.04 (x2.94)	x1.10	2.78 (x2.03)	x1.08	2.32 (x1.69)	2.69 (x27.9)	x1.14	1.02 (x0.52)	x1.04	0.791 (x0.40)
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol JIT O3	0.579 (x0.28)	x1.43	2.02 (x1.47)	x1.03	1.4 (x1.02)	0.211 (x2.20)	x1.10	0.778 (x0.40)	x1.00	0.501 (x0.26)
5	d d~ > Z + (5-1)*g	reference	AmpliCol	2.32		4.36		N/A	0.0113		6.57		N/A
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol JIT O1	1.35 (x0.58)	x2.16	5.59 (x1.28)	x1.01	4.33 (x0.99)	0.417 (x36.8)	x1.37	3.03 (x0.46)	x1.04	2.23 (x0.34)
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol ASM O3	4.93 (x2.13)	x1.27	17.5 (x4.02)	x1.08	16.3 (x3.74)	2.42 (x213)	x1.17	10.6 (x1.61)	x1.11	10 (x1.53)
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol C++ O3	18.1 (x7.80)	x1.09	8.57 (x1.97)	x1.06	7.33 (x1.68)	9.16 (x809)	x1.09	4.52 (x0.69)	x1.05	4.09 (x0.62)
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol JIT O3	1.37 (x0.59)	x2.36	5.81 (x1.33)	x1.03	4.41 (x1.01)	0.378 (x33.4)	x1.23	3.06 (x0.47)	x1.04	2.24 (x0.34)
6	d d~ > Z + (6-1)*g	reference	AmpliCol	3.19		13.9		N/A	0.107		41.5		N/A
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O1	3.23 (x1.01)	x2.85	17 (x1.22)	x1.23	13.9 (x1.00)	1.58 (x14.8)	x1.18	33.9 (x0.82)	x1.03	30.5 (x0.73)
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol ASM O3	9.96 (x3.13)	x1.35	51.3 (x3.68)	x1.10	48.4 (x3.47)	4.89 (x45.8)	x1.09	74.8 (x1.80)	x1.09	70.6 (x1.70)
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol C++ O3	50.3 (x15.8)	x1.07	27.6 (x1.98)	x1.07	25.1 (x1.80)	74.8 (x700)	x1.05	57.2 (x1.38)	x1.00	56.6 (x1.36)
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O3	3.25 (x1.02)	x2.75	17.2 (x1.23)	x1.11	14.6 (x1.05)	1.62 (x15.1)	x1.18	35.9 (x0.86)	x1.04	32 (x0.77)

Reproduce the $n = 6$ JIT O1 selected-flow entry with the following generation and timing commands.

```
env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol generate-process --model assets/models/json/sm/sm.json 'd d~ > z g g g g g' docs/.z_performance_ufo_sm_outputs/manual/n6/jit_o1 --replace --n_cores 5 --color-accuracy lc --batch-size 64
      --symbolica-n-cores 5 --symbolica-output-chunk-size 128 --symbolica-output-chunk-strategy uniform --symbolica-stage-local-parameter-layout --symbolica-iterations 10 --symbolica-max-horner-scheme-variables 1000
      --symbolica-max-common-pair-cache-entries 5000000 --symbolica-max-common-pair-distance 1000 --lc-sector-ids 0 --reference-color-order 2,4,5,6,7,8,1,3 --symbolica-evaluator-backend jit --symbolica-jit-optimization-level 1
env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol time-process --target-runtime 10 --batch-size 64 --json docs/.z_performance_ufo_sm_outputs/manual/n6/jit_o1
```

pyAmpliCol model source: **`external UFO/JSON SM`**. Here n is final-state multiplicity. Each block reports generation seconds and wall/evaluator microseconds per point: one selected flow with the helicity sum on the left, all flows at one fixed helicity on the right. Wall measures Rusticol's public **`evaluate`** call; eval separately profiles evaluator plus input packing. Both use batch 64, stage-local inputs, ten Horner iterations, and a ten-second timing target; output chunks are 128 selected and 8192 all-flow. The green O1 row is the measured default: repeated O3 timings showed no significant Z-family gain. ASM remains a scalar AArch64 diagnostic. The driver keeps generated artifacts under **`docs/.z_performance_ufo_sm_outputs`**. AmpliCol's generated-library time is reported under wall; its separate eval metric is N/A. Its **`vs blt-in`** cells are left blank. In this external-model table, each **`vs blt-in`** entry is the adjacent generation- or wall-time ratio to the matching built-in-SM row at the same multiplicity, backend, and flow/helicity workload; values below one mean that the external model is faster.

14 External-SM NLC Performance Matrix

pyAmpliCol model source: `external UFO/JSON SM`. Cell format: generation time above runtime per phase-space point; slots are AmpliCol and pyAmpliCol JIT O1; pyAmpliCol C++ O3 is also shown only through $n = 3$. AmpliCol uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

<table>
<tr><th>ID</th><th>base process</th><th>n=1</th><th>n=2</th><th>n=3</th></tr>
<tr><td>1</td><td>$d\bar{d} \to Z + (n-1)g$</td><td>1.983 (x0.094) (N/A)
0.084 (x1.88) (x0.52) (N/A)</td><td>2.07 (x0.103) (N/A)
0.228 (x1.53) (x0.708) (N/A)</td><td>2.137 (x0.15) (N/A)
1.337 (x1.13) (x0.701) (N/A)</td></tr>
<tr><td>2</td><td>$u\bar{d} \to W^+ + (n-1)g$</td><td>2.049 (x0.092) (N/A)
0.0539 (x2.31) (x0.546) (N/A)</td><td>2.059 (x0.097) (N/A)
0.132 (x1.91) (x0.782) (N/A)</td><td>2.086 (x0.145) (N/A)
0.742 (x1.34) (x0.787) (N/A)</td></tr>
<tr><td>3</td><td>$d\bar{d} \to e^+e^- + (n-2)g$</td><td>N/A</td><td>2.07 (x0.1) (N/A)
0.178 (x1.18) (x0.453) (N/A)</td><td>2.125 (x0.12) (N/A)
0.405 (x0.994) (x0.515) (N/A)</td></tr>
<tr><td>4</td><td>$u\bar{d} \to e^+\nu_e + (n-2)g$</td><td>N/A</td><td>2.043 (x0.1) (N/A)
0.0968 (x1.26) (x0.368) (N/A)</td><td>2.085 (x0.154) (N/A)
0.185 (x1.21) (x0.483) (N/A)</td></tr>
<tr><td>5</td><td>$d\bar{d} \to ZZ + (n-2)g$</td><td>N/A</td><td>2.1 (x0.105) (N/A)
0.324 (x1.31) (x0.641) (N/A)</td><td>2.152 (x0.145) (N/A)
0.991 (x1.12) (x0.7) (N/A)</td></tr>
<tr><td>6</td><td>$gg \to t\bar{t} + (n-2)g$</td><td>N/A</td><td>2.062 (x0.114) (N/A)
0.677 (x1.16) (x0.626) (N/A)</td><td>2.384 (x0.26) (N/A)
6.593 (x1.29) (x0.897) (N/A)</td></tr>
<tr><td>7</td><td>$d\bar{d} \to t\bar{t} + (n-2)g$</td><td>N/A</td><td>2.114 (x0.099) (N/A)
0.479 (x0.884) (x0.337) (N/A)</td><td>2.337 (x0.157) (N/A)
2.369 (x0.994) (x0.578) (N/A)</td></tr>
<tr><td>8</td><td>$gg \to gg + (n-2)g$</td><td>N/A</td><td>2.253 (x0.133) (N/A)
3.216 (x0.452) (x0.308) (N/A)</td><td>3.166 (x0.433) (N/A)
48.65 (x0.634) (x0.428) (N/A)</td></tr>
<tr><td>9</td><td>$d\bar{d} \to ZZZ + (n-3)g$</td><td>N/A</td><td>N/A</td><td>2.553 (x0.151) (N/A)
1.904 (x1.16) (x0.802) (N/A)</td></tr>
<tr><td>10</td><td>$d\bar{d} \to e^+e^-ZH + (n-4)g$</td><td>N/A</td><td>N/A</td><td>N/A</td></tr>
<tr><td>11</td><td>$d\bar{d} \to t\bar{t}ZH + (n-4)g$</td><td>N/A</td><td>N/A</td><td>N/A</td></tr>
<tr><td>12</td><td>$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$</td><td>N/A</td><td>N/A</td><td>N/A</td></tr>
<tr><td>13</td><td>$d\bar{d} \to u\bar{u}\, s\bar{s} + (n-4)g$</td><td>N/A</td><td>N/A</td><td>N/A</td></tr>
<tr><td>14</td><td>$d\bar{d} \to u\bar{u}\, s\bar{s}\, c\bar{c} + (n-6)g$</td><td>N/A</td><td>N/A</td><td>N/A</td></tr>
<tr><td colspan="2">summary: gen</td><td>1.98| 2.05| 2.02| 2.02
x0.092| x0.094| x0.093| x0.093| x0.093</td><td>2.04| 2.25| 2.1| 2.07
x0.097| x0.133| x0.106| x0.101| x0.106</td><td>2.08| 3.17| 2.34| 2.15
x0.12| x0.433| x0.191| x0.151| x0.202</td></tr>
<tr><td colspan="2">summary: run</td><td>0.0539| 0.084| 0.0689| 0.0689
x1.88| x2.31| x2.10| x2.10| x2.05</td><td>0.0968| 3.22| 0.666| 0.276
x0.452| x1.91| x1.21| x1.22| x0.754</td><td>0.185| 48.6| 7.02| 1.34
x0.634| x1.34| x1.10| x1.13| x0.762</td></tr>
</table>

External-SM NLC Performance Matrix (continued)

ID	base process	n=4		n=5		n=6
1	$d\bar{d} \to Z + (n-1)g$	2.588 12.05	(x0.361) (x1.44) (x0.951)	4.953 154	(x2.00) (x1.53) (x1.24)	N/A
2	$u\bar{d} \to W^+ + (n-1)g$	2.336 7.009	(x0.269) (x1.53) (x0.976)	3.727 94.09	(x1.72) (x1.79) (x1.50)	N/A
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.24 2.062	(x0.154) (x0.714) (x0.474)	3.226 17.35	(x0.311) (x0.755) (x0.572)	N/A
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2.14 0.97	(x0.12) (x0.733) (x0.429)	2.752 8.798	(x0.209) (x0.711) (x0.527)	N/A
5	$d\bar{d} \to ZZ + (n-2)g$	2.337 5.508	(x0.24) (x1.16) (x0.84)	4.248 48.28	(x0.829) (x1.45) (x1.20)	N/A
6	$gg \to t\bar{t} + (n-2)g$	4.299 91.22	(x1.12) (x1.74) (x1.39)	29.13 1.5e+03	(x3.14) (x1.89) (x1.73)	N/A
7	$d\bar{d} \to t\bar{t} + (n-2)g$	3.9 23.63	(x0.35) (x1.74) (x1.01)	16.75 299	(x1.08) (x1.86) (x1.49)	N/A
8	$gg \to gg + (n-2)g$	13.65 828	(x1.56) (x0.874) (x0.759)	168 1.91e+04	(x4.44) (x0.687) (x0.592)	N/A
9	$d\bar{d} \to ZZZ + (n-3)g$	3.165 5.902	(x0.182) (x1.04) (x0.743)	6.75 30.76	(x0.345) (x1.33) (x1.15)	N/A
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	4.008 1.492	(x0.086) (x1.08) (x0.733)	4.902 3.254	(x0.092) (x0.913) (x0.685)	N/A
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	2.867 3.852	(x0.158) (x0.959) (x0.683)	4.376 21.19	(x0.384) (x1.31) (x1.15)	N/A
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	4.158 2.835	(x0.094) (x0.557) (x0.396)	5.283 6.219	(x0.099) (x0.471) (x0.357)	N/A
13	$d\bar{d} \to u\bar{u}\, s\bar{s} + (n-4)g$	N/A N/A	0.42 (3.6)	N/A		N/A
14	$d\bar{d} \to u\bar{u}\, s\bar{s}\, c\bar{c} + (n-6)g$	N/A		N/A		N/A

	n=4 min	max	avg	med	sum	n=5 min	max	avg	med	sum	n=6
summary: gen	2.14	13.7	3.97	3.02		2.75	168	21.1	4.93		N/A
	x0.086	x1.56	x0.391	x0.211	x0.67	x0.092	x4.44	x1.22	x0.606	x3.47	N/A
summary: run	0.97	828	82	5.71		3.25	1.91e4	1.78e3	39.5		N/A
	x0.557	x1.74	x1.13	x1.06	x0.988	x0.471	x1.89	x1.22	x1.32	x0.803	N/A

Each cell is raw-library AmpliCol and pyAmpliCol JIT O1; the optional pyAmpliCol C++ O3 slot is shown only through $n = 3$. Entries are generation seconds above runtime microseconds per point; pyAmpliCol runtime shows coloured wall and black pure-evaluator ratios, or absolute time without an AmpliCol reference. The pyAmpliCol entries use the compiled `external UFO/JSON SM` source; the AmpliCol columns are the same matched external reference used by the built-in-SM table. Green/orange/red means below one/below two/at least two. Summaries are `min|max|avg|med|sum`; `sum` compares paired totals. A **VALIDATION FAILED** marker means the same-point difference exceeds the tolerance recorded for that validation result.

Settings. `docs/result_matrix.py` (`-color-accuracy nlc`) writes this table and its JSON cache. AmpliCol uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`. One raw evaluator call returns every stored helicity amplitude for a phase-space point; the probe then applies the requested sparse NLC/full colour contractions without reevaluating those amplitudes. pyAmpliCol uses Rusticol double precision, batch size 64, output chunk size 128, Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1, and `-target-runtime 10` are used. C++ O3 is displayed only through $n = 3$; its generation alone is subject to the 15-minute compile cap.

Fortran reference gaps. 1 completed pyAmpliCol cells have no direct Fortran colour reference because that path stops above two quark lines; they are shown as absolute timings.

C++ O3 gaps. 19 optional cells were not run and 0 exceeded the 15-minute compile budget; completed AmpliCol/JIT data remain shown.

15 External-SM Full-Colour Performance Matrix

pyAmpliCol model source: `external UFO/JSON SM`. Cell format: generation time above runtime per phase-space point; slots are AmpliCol and pyAmpliCol JIT O1; pyAmpliCol C++ O3 is also shown only through $n = 3$. AmpliCol uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

<table>
<tr><th>ID</th><th>base process</th><th>n=1</th><th>n=2</th><th>n=3</th></tr>
<tr><td>1</td><td>$d\bar{d} \to Z + (n-1)g$</td><td>2.046 (x0.091) (N/A)
0.0969 (x1.68) (x0.471) (N/A)</td><td>2.083 (x0.103) (N/A)
0.271 (x1.29) (x0.593) (N/A)</td><td>2.11 (x0.163) (N/A)
1.546 (x1.03) (x0.631) (N/A)</td></tr>
<tr><td>2</td><td>$u\bar{d} \to W^+ + (n-1)g$</td><td>2.027 (x0.095) (N/A)
0.06 (x2.09) (x0.491) (N/A)</td><td>2.058 (x0.103) (N/A)
0.147 (x1.73) (x0.697) (N/A)</td><td>2.087 (x0.145) (N/A)
0.85 (x1.18) (x0.688) (N/A)</td></tr>
<tr><td>3</td><td>$d\bar{d} \to e^+e^- + (n-2)g$</td><td>N/A</td><td>2.091 (x0.095) (N/A)
0.196 (x1.05) (x0.401) (N/A)</td><td>2.134 (x0.128) (N/A)
0.449 (x0.891) (x0.461) (N/A)</td></tr>
<tr><td>4</td><td>$u\bar{d} \to e^+\nu_e + (n-2)g$</td><td>N/A</td><td>2.053 (x0.092) (N/A)
0.11 (x1.10) (x0.318) (N/A)</td><td>2.096 (x0.107) (N/A)
0.203 (x1.09) (x0.432) (N/A)</td></tr>
<tr><td>5</td><td>$d\bar{d} \to ZZ + (n-2)g$</td><td>N/A</td><td>2.115 (x0.101) (N/A)
0.384 (x1.09) (x0.541) (N/A)</td><td>2.151 (x0.144) (N/A)
1.088 (x1.03) (x0.646) (N/A)</td></tr>
<tr><td>6</td><td>$gg \to t\bar{t} + (n-2)g$</td><td>N/A</td><td>2.094 (x0.11) (N/A)
0.807 (x0.973) (x0.51) (N/A)</td><td>2.385 (x0.244) (N/A)
8.11 (x1.27) (x0.753) (N/A)</td></tr>
<tr><td>7</td><td>$d\bar{d} \to t\bar{t} + (n-2)g$</td><td>N/A</td><td>2.084 (x0.099) (N/A)
0.635 (x0.658) (x0.259) (N/A)</td><td>2.329 (x0.165) (N/A)
2.986 (x0.799) (x0.469) (N/A)</td></tr>
<tr><td>8</td><td>$gg \to gg + (n-2)g$</td><td>N/A</td><td>2.246 (x0.132) (N/A)
4.149 (x0.349) (x0.252) (N/A)</td><td>3.133 (x0.474) (N/A)
63.18 (x0.478) (x0.343) (N/A)</td></tr>
<tr><td>9</td><td>$d\bar{d} \to ZZZ + (n-3)g$</td><td>N/A</td><td>N/A</td><td>2.559 (x0.156) (N/A)
2.032 (x1.11) (x0.763) (N/A)</td></tr>
<tr><td>10</td><td>$d\bar{d} \to e^+e^-ZH + (n-4)g$</td><td>N/A</td><td>N/A</td><td>N/A</td></tr>
<tr><td>11</td><td>$d\bar{d} \to t\bar{t}ZH + (n-4)g$</td><td>N/A</td><td>N/A</td><td>N/A</td></tr>
<tr><td>12</td><td>$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$</td><td>N/A</td><td>N/A</td><td>N/A</td></tr>
<tr><td>13</td><td>$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$</td><td>N/A</td><td>N/A</td><td>N/A</td></tr>
<tr><td>14</td><td>$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$</td><td>N/A</td><td>N/A</td><td>N/A</td></tr>
<tr><td colspan="2">summary: gen</td><td>2.03| 2.05| 2.04| 2.04
x0.091| x0.095| x0.093| x0.093| x0.093</td><td>2.05| 2.25| 2.1| 2.09
x0.092| x0.132| x0.104| x0.102| x0.105</td><td>2.09| 3.13| 2.33| 2.15
x0.107| x0.474| x0.192| x0.156| x0.205</td></tr>
<tr><td colspan="2">summary: run</td><td>0.06| 0.0969| 0.0784| 0.0784
x1.68| x2.09| x1.89| x1.89| x1.84</td><td>0.11| 4.15| 0.837| 0.328
x0.349| x1.73| x1.03| x1.07| x0.597</td><td>0.203| 63.2| 8.94| 1.55
x0.478| x1.27| x0.986| x1.03| x0.615</td></tr>
</table>

External-SM Full-Colour Performance Matrix (continued)

ID	base process	n=4		n=5		n=6
1	$d\bar{d} \to Z + (n-1)g$	2.615	(x0.356)	4.981	(x2.32)	N/A
		14.86	(x1.16) (x0.769)	208	(x1.22) (x0.945)	
2	$u\bar{d} \to W^+ + (n-1)g$	2.334	(x0.291)	3.718	(x1.89)	N/A
		8.097	(x1.27) (x0.852)	122	(x1.59) (x1.14)	
3	$d\bar{d} \to e^+e^- + (n-2)g$	2.211	(x0.143)	3.252	(x0.32)	N/A
		2.355	(x0.688) (x0.439)	20.64	(x0.692) (x0.498)	
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	2.135	(x0.128)	2.773	(x0.213)	N/A
		1.126	(x0.697) (x0.402)	9.978	(x0.647) (x0.462)	
5	$d\bar{d} \to ZZ + (n-2)g$	2.353	(x0.239)	4.413	(x0.802)	N/A
		6.147	(x1.05) (x0.769)	56.7	(x1.24) (x1.19)	
6	$gg \to t\bar{t} + (n-2)g$	4.316	(x1.13)	28.64	(x3.43)	N/A
		129	(x1.28) (x0.946)	3.05e+03	(x1.21) (x0.771)	
7	$d\bar{d} \to t\bar{t} + (n-2)g$	3.93	(x0.333)	16.78	(x1.80)	N/A
		32.46	(x0.876) (x0.837)	510	(x2.67) (x0.887)	
8	$gg \to gg + (n-2)g$	13.65	(x1.84)	168	(x8.46)	N/A
		1.43e+03	(x0.6) (x0.444)	5.47e+04	(x2.01) (x0.468)	
9	$d\bar{d} \to ZZZ + (n-3)g$	3.121	(x0.19)	6.556	(x0.364)	N/A
		6.077	(x0.959) (x0.71)	32.49	(x1.25) (x1.17)	
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	3.994	(x0.089)	4.835	(x0.178)	N/A
		1.532	(x1.07) (x0.731)	3.345	(x1.55) (x1.09)	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	2.794	(x0.154)	4.391	(x0.651)	N/A
		4.357	(x0.86) (x0.604)	23.72	(x3.23) (x3.22)	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	4.13	(x0.081)	5.311	(x0.092)	N/A
		2.892	(x0.526) (x0.385)	6.453	(x0.454) (x0.36)	
13	$d\bar{d} \to u\bar{u}\, s\bar{s} + (n-4)g$	N/A	0.399	N/A		N/A
		N/A	(3.12)			
14	$d\bar{d} \to u\bar{u}\, s\bar{s}\, c\bar{c} + (n-6)g$	N/A		N/A		N/A
summary: gen		2.14| 13.6| 3.97| 2.96		2.77| 168| 21.1| 4.91		N/A
		x0.081| x1.84| x0.415| x0.215| x0.752		x0.092| x8.46| x1.71| x0.726| x6.23		N/A
summary: run		1.13| 1.43e3| 136| 6.11		3.35| 5.47e4| 4.9e3| 44.6		N/A
		x0.526| x1.28| x0.919| x0.918| x0.671		x0.454| x3.23| x1.48| x1.25| x1.97		N/A

Each cell is raw-library AmpliCol and pyAmpliCol JIT O1; the optional pyAmpliCol C++ O3 slot is shown only through $n = 3$. Entries are generation seconds above runtime microseconds per point; pyAmpliCol runtime shows coloured wall and black pure-evaluator ratios, or absolute time without an AmpliCol reference. The pyAmpliCol entries use the compiled `external UFO/JSON SM` source; the AmpliCol columns are the same matched external reference used by the built-in-SM table. Green/orange/red means below one/below two/at least two. Summaries are `min|max|avg|med|sum`; `sum` compares paired totals. A **VALIDATION FAILED** marker means the same-point difference exceeds the tolerance recorded for that validation result.

Settings. `docs/result_matrix.py` (`-color-accuracy full`) writes this table and its JSON cache. AmpliCol uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`. One raw evaluator call returns every stored helicity amplitude for a phase-space point; the probe then applies the requested sparse NLC/full colour contractions without reevaluating those amplitudes. pyAmpliCol uses Rusticol double precision, batch size 64, output chunk size 128, Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1, and `-target-runtime 10` are used. C++ O3 is displayed only through $n = 3$; its generation alone is subject to the 15-minute compile cap.

Fortran reference gaps. 1 completed pyAmpliCol cells have no direct Fortran colour reference because that path stops above two quark lines; they are shown as absolute timings.

C++ O3 gaps. 19 optional cells were not run and 0 exceeded the 15-minute compile budget; completed AmpliCol/JIT data remain shown.

16 Colorless UFO Models

The bundled `scalars` and `scalar_gravity` examples exercise the same source and process pipeline without Standard-Model-specific kernels. Their UFO directories and loader JSON serializations compile to identical canonical IR fingerprints; an exact-version compiled model is then the common input to process generation. Because every particle is a colour singlet, LC, NLC, and full colour all reduce to the unit colour metric. A dedicated runtime regression checks that equality together with UFO/JSON/compiled-source parity.

The contact-oracle ladder isolates direct vertices through valence ten. Each contact is decomposed into balanced, non-propagating auxiliary currents, and the ten-point result is independently checked against an eager contraction of the original n-ary UFO tensor. A second, uncapped ladder retains every allowed scalar tree and internal propagator; it completes through seven final scalars, with the largest row using the documented one-hour generation allowance. For scalar gravity, a massless graviton helicity tensor is built from equal-helicity transverse vector products:

$$\epsilon^{\mu\nu}_{\pm 2} = \epsilon^{\mu}_{\pm}\epsilon^{\nu}_{\pm}.$$

The resulting states are symmetric, transverse, traceless, and normalized by the formula-level arbitrary-precision tests. Large tensor components stay in the Symbolica function map described above, avoiding repeated expression expansion during stage construction.

The zero-current and identical-current warmups use the same ten points. When zero filtering leaves the DAG unchanged, pyAmpliCol now reuses its current signatures for merging; reuse is disabled if filtering changes the graph or the estimated cache exceeds 64 MiB. This removes a duplicated spin-2 contraction without weakening either numerical rewrite check or changing high-multiplicity SM memory policy.

Massless Scalar Contact Model Validation Ladder

n	process	gen [s]	JIT [s]	currents	interactions	value	eval [μs]	wall [μs]	rel. 50d
2	$\phi_0\,\phi_0 \to \phi_0\,\phi_0$	0.1813	0.002478	8	6	`0.5`	0.008498	0.1089	0
3	$\phi_0\,\phi_0 \to \phi_0\,\phi_0\,\phi_0$	0.2673	0.002408	18	18	`0.16666667`	0.01555	0.1525	0
4	$\phi_0\,\phi_0 \to \phi_0\,\phi_0\,\phi_0\,\phi_0$	0.6286	0.003183	37	60	`0.041666667`	0.03094	0.3203	0
5	$\phi_0\,\phi_0 \to \phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0$	1.974	0.00389	78	170	`0.0083333333`	0.05994	0.5024	0
6	$\phi_0\,\phi_0 \to \phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0$	6.585	0.006382	142	413	`0.0013888889`	0.1235	1.124	0
7	$\phi_0\,\phi_0 \to \phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0$	21.4	0.2644	262	1,022	`0.0001984127`	0.2666	1.437	0
8	$\phi_0\,\phi_0 \to \phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0$	74.72	0.203	491	2,538	`2.4801587e-05`	0.7979	4.574	0

UFO conversion: `0.3598 s`; JSON conversion: `0.3503 s`; compiled load: `0.07456 s`.

Contact oracle. The scalar ladder explicitly sets `-max-coupling-order QCD=1`. With $\lambda = 1$ this retains only the direct $(n+2)$-point vertex, so no propagator denominator enters and the identical-final-state normalization gives $1/n!$. It tests generic high-rank contact decomposition, not the sum of all scalar-model tree diagrams; the uncapped all-tree ladder below provides that complementary test. Values use the retained RAMBO point generated with seed 101 at $\sqrt{s} = 1\,\mathrm{TeV}$. Generation starts from the compiled model and uses SymJIT O1, batch/chunk size 128 and a 30 GB watchdog. The scalar rows use the ordinary five-minute generation cap. The eval column isolates generated evaluators; wall includes Rusticol orchestration. Runtime uses double precision and `-target-runtime 10`; the final column compares the same point with a 50-digit reevaluation. Superseded process directories are archived beside the active output.

Massless Scalar All-Tree Model Validation Ladder

n	process	gen [s]	JIT [s]	currents	interactions	value	eval [μs]	wall [μs]	rel. 50d
2	$\phi_0\,\phi_0 \to \phi_0\,\phi_0$	0.2008	0.01201	18	24	0.50001868	0.02333	0.1423	0
3	$\phi_0\,\phi_0 \to \phi_0\,\phi_0\,\phi_0$	0.395	0.008669	72	271	0.16662817	0.1028	0.3992	1.67×10^{-16}
4	$\phi_0\,\phi_0 \to \phi_0\,\phi_0\,\phi_0\,\phi_0$	2.628	0.4168	333	1,905	0.041644958	0.6876	1.6	0
5	$\phi_0\,\phi_0 \to \phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0$	12.92	1.396	1,575	11,074	0.0084164373	4.172	7.792	8.24×10^{-16}
6	$\phi_0\,\phi_0 \to \phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0$	80.58	18.58	7,532	114,303	0.0013863002	60.33	84.95	1.56×10^{-16}
7	$\phi_0\,\phi_0 \to \phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0\,\phi_0$	382	236.6	32,176	684,889	0.0001982244	310.3	387.7	6.84×10^{-16}

UFO conversion: `0.3598 s`; JSON conversion: `0.3503 s`; compiled load: `0.07456 s`.

Complete scalar trees. These entries omit the coupling-order cap and therefore sum the direct contact term with every allowed multi-vertex tree and internal scalar propagator. They use the same model parameters and phase-space convention as the contact oracle above. Values use the retained RAMBO point generated with seed 101 at $\sqrt{s} = 1\,\mathrm{TeV}$. Generation starts from the compiled model and uses SymJIT O1, batch/chunk size 128 and a 30 GB watchdog. The ordinary generation cap is five minutes; the seven-scalar entry is allowed one hour. The eval column isolates generated evaluators; wall includes Rusticol orchestration. Runtime uses double precision and `-target-runtime 10`; the final column compares the same point with a 50-digit reevaluation. Superseded process directories are archived beside the active output.

Scalar-Gravity Model Validation Ladder

n	process	gen [s]	JIT [s]	currents	interactions	value	eval [μs]	wall [μs]	rel. 50d
2	$\phi_0\,\phi_0 \to G\,G$	0.2367	0.01262	14	12	1.1963663e+09	0.1835	0.5659	1.99×10^{-16}
3	$\phi_0\,\phi_0 \to G\,G\,G$	73.75	42.66	104	292	5.2311638e+11	1074	1117	1.40×10^{-15}
4	$\phi_0\,\phi_0 \to G\,G\,G\,G$	880.6	603.3	378	2,168	6.9756678e+13	3.53e+04	3.53e+04	3.58×10^{-15}

UFO conversion: `33.29 s`; JSON conversion: `33.73 s`; compiled load: `0.1585 s`.

Full-model ladder. The scalar-gravity entries retain the complete tree-level model, including internal scalar and graviton propagators. Values use the retained RAMBO point generated with seed 101 at $\sqrt{s} = 1\,\mathrm{TeV}$. Generation starts from the compiled model and uses SymJIT O1, batch/chunk size 128 and a 30 GB watchdog. The ordinary generation cap is five minutes; the four-graviton entry is allowed one hour. The eval column isolates generated evaluators; wall includes Rusticol orchestration. Runtime uses double precision and `-target-runtime 10`; the final column compares the same point with a 50-digit reevaluation. Superseded process directories are archived beside the active output.

17 Why This Matches AmpliCol but Remains Python-Friendly

The Python layer owns graph construction, process metadata, deterministic phase-space steering, artifact generation, evaluator caching, and validation against AmpliCol. It does not perform the hot current recursion component by component in production. The two production execution routes are now:

$$\begin{aligned}\text{Python staged route:}\quad & \text{source fill} \to \text{bounded stage evaluators} \to \text{amplitude/raw-sum evaluator},\\ \text{Rusticol route:}\quad & \text{one PyO3 call} \to \text{Rust source fill and staged sweep} \to \text{amplitude/raw-sum evaluator}.\end{aligned}$$

Thus pyAmpliCol keeps AmpliCol's essential optimization - shared currents across helicity parents and a topological current-table sweep - while replacing generated Fortran subroutines by saved Spenso/Symbolica evaluator artifacts. The `rusticol` path removes the remaining Python orchestration cost by loading the same process directory and executing the sweep in Rust.

The old fully monolithic compiled-DAG route is now deprecated and kept only as reference code. Its alias-based design was useful for exploring whole-graph compilation, but the production target is the staged DAG: Python generation plus a Rust eager-DAG runtime over saved Symbolica stage evaluators. This keeps the runtime schedule close to AmpliCol's current-table library while leaving the graph compiler model-driven and process-generic.